



ML-BENCH: Evaluating Large Language Models and Agents for Machine Learning Tasks on Repository-Level Code

Xiangru Tang*♣ Yuliang Liu*♣ Zefan Cai*♡ Yanjun Shao*♣ Junjie Lu♣ Yichi Zhang♡
 Zexuan Deng♣ Helan Hu♡ Kaikai An♡ Ruijun Huang Shuzheng Si♡ Sheng Chen♣
 Haozhe Zhao♡ Liang Chen♡ Yan Wang Tianyu Liu♡ Zhiwei Jiang♣ Baobao Chang♡
 Yin Fang Yujia Qin Wangchunshu Zhou Yilun Zhao♣ Arman Cohan♣ Mark Gerstein♣

♣ Yale University ♣ Nanjing University ♡ Peking University

xiangru.tang@yale.edu, pi@gersteinlab.org

Abstract

Despite Large Language Models (LLMs) like GPT-4 achieving impressive results in function-level code generation, they struggle with repository-scale understanding (e.g., coming up with the right arguments for calling routines), requiring a deeper comprehension of complex file interactions. Also, recently, people have developed LLM agents that attempt to interact with repository code (e.g., compiling and evaluating its execution), prompting the need to evaluate their performance. These gaps have motivated our development of ML-BENCH, a benchmark rooted in real-world programming applications that leverage existing code repositories to perform tasks. Addressing the need for LLMs to interpret long code contexts and translate instructions into precise, executable scripts, ML-BENCH encompasses annotated **9,641** examples across **18** GitHub repositories, challenging LLMs to accommodate user-specified arguments and documentation intricacies effectively. To evaluate both LLMs and AI agents, two setups are employed: ML-LLM-BENCH for assessing LLMs’ text-to-code conversion within a predefined deployment environment, and ML-AGENT-BENCH for testing autonomous agents in an end-to-end task execution within a Linux sandbox environment. Our findings indicate that while GPT-4o leads with a Pass@5 rate surpassing **50%**, there remains significant scope for improvement, highlighted by issues such as hallucinated outputs and difficulties with bash script generation. Notably, in the more demanding ML-AGENT-BENCH, GPT-4o achieves a **76.47%** success rate, reflecting the efficacy of iterative action and feedback in complex task resolution. Our resources, including code, data, and models, are publicly available under the MIT license.



Code

<https://github.com/gersteinlab/ML-bench>



Benchmark

<https://hf.co/datasets/super-dainiu/ml-bench>



Webpage

<https://ml-bench.github.io/>

*Contribute equally.

Preprint. Under review.

1 Introduction

Large Language Models (LLMs) like GPT-4 have demonstrated remarkable prowess, reaffirming their state-of-the-art status on function-level code generation within widely recognized benchmarks such as HumanEval, MBPP, and APPS [Austin et al., 2021, Chen et al., 2021, Hendrycks et al., 2021b, Li et al., 2022]. Nevertheless, their capability wanes at the repository scale, which poses the multifaceted challenge of processing extensive contexts and navigating the intricate web of file interdependencies. This reveals the first gap in benchmarking - *the absence of thorough assessments to test LLMs' abilities in complex repository-level deployment tasks*, such as interacting with functions in different files. Moreover, the emergence of recent AI agents capable of executing and iteratively refining code marks a significant advancement [Shinn et al., 2023, Yang et al., 2024, Zhou et al., 2023]. Yet, a second gap persists in benchmarking such agents: *the need for a rigorous evaluation environment* that can effectively measure an agent's skill in the initialization and utilization of software repositories.

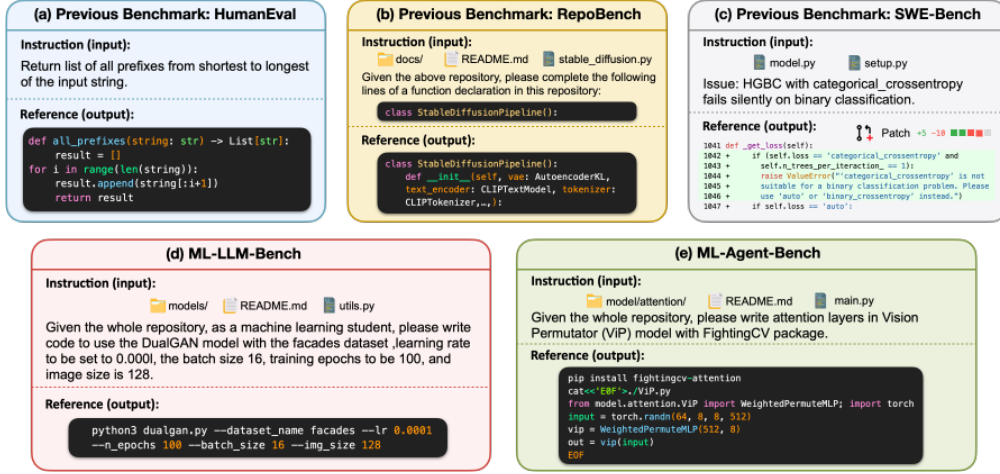


Figure 1: Examples of ML-BENCH compared with existing code benchmarks HumanEval [Chen et al., 2021], RepoBench [Liu et al., 2023], and SWE-bench [Jimenez et al., 2024]. In ML-BENCH, (1) models must **take repository-level code as input**, and (2) based on their understanding of the repository, models are required to compose **new code segments that do not exist** within the original repository.

We introduce ML-BENCH based on common real-world programming workflows, often using existing repositories as libraries to accomplish tasks. ML-BENCH seeks to test whether LLMs can generate executable code to invoke specific files or functions in a repository with appropriate arguments based on given instructions. For instance, it might assess if an LLM can generate a command line to utilize `txt2img.py` from an image generation model repository with parameters such as `ckpt` to produce an image based on a text description, e.g. `python txt2img.py --prompt "a girl riding a horse" --ckpt SD2_1_v_model.ckpt`. To effectively address these tasks, LLMs need to understand code at the repository level, which often involves processing a long context. In addition, they need to accurately configure parameters when executing program files. For instance, LLMs must discern how to input values from user instructions correlate with flags found across code in various files, see Figure 1. Another crucial aspect of this process is understanding documentation—especially README files—which typically includes comprehensive instructions on how to employ the library, complete with task examples and arguments selection guidelines. In practice, user-specified arguments don’t always match the examples provided in README files, thus requiring LLMs to modify arguments intelligently to meet specific user needs. ML-BENCH features **9,641** samples from **18** ML GitHub repositories, depicted in Figure 3.

To better assess the abilities of LLMs versus autonomous AI agents, we present two distinct testing setups: ML-LLM-BENCH and ML-AGENT-BENCH, as shown in Figure 2. (1) For LLMs, we evaluate the model’s capacity to complete the task of calling code within our *configured deployment environment*. It benchmarks their competence in *translating text instructions to simple bash code with clearly defined parameters*. (2) However, a more arduous challenge lies in the end-to-end execution of tasks, starting from scratch. This involves initiating the code environment for a specific repository, where common pitfalls of environment setup, such as missing datasets or uninstalled packages, might occur. To evaluate agents in such a setup, we introduce a secure Linux sandbox

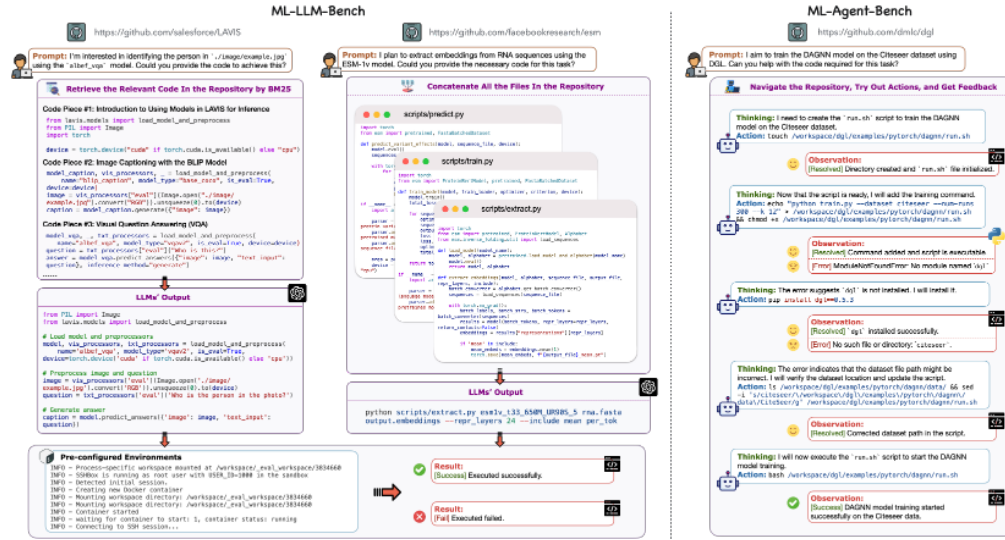


Figure 2: The workflow of ML-BENCH, including ML-LLM-Bench and ML-Agent-Bench. In ML-LLM-Bench, LLMs generate Python code or Bash scripts based on the prompt. The input to the LLMs could be code retrieved from a repository based on the prompt or a direct concatenation of all files. Their performance is evaluated within a pre-configured environment. Conversely, in ML-Agent-Bench, the agent must autonomously set up the environment and download necessary datasets to accomplish the task.

environment, where agents can *iteratively execute commands and code blocks to obtain feedback*. The agent’s actions involve multiple attempts, from reading files and understanding the repository to installing dependencies, preparing datasets, and finally writing bash code that calls the repository, thus emulating the full workflow of a human coder.

- **ML-LLM-Bench:** This benchmark component includes tasks that require generating both bash scripts and Python code. The diversity in task types reflects the varied nature of ML workflows, where both scripting and programming play essential roles.
- **ML-Agent-Bench:** In this more complex setup, agents must interact with the environment using a combination of bash commands and Python code with tools such as Jupyter Notebooks. This approach closely mimics ML practitioners’ workflow, who often switch between command-line operations and code execution in interactive environments.

In our evaluation experiment on ML-LLM-BENCH, we observe that GPT-4o outperforms other LLMs, being the sole model to surpass the **50%** threshold in the Pass@5 metric (success rate within five tries). It is noteworthy that in the same test set, our annotators’ performance—computer science graduate students—stood at a success rate of **86.76%**, with **59** out of **68** examples correctly executed. This indicates substantial room for improvement in current LLMs. However, the models did show performance improvements following instruction tuning on the training data (**8.85**→**15.76** for CodeLlama).

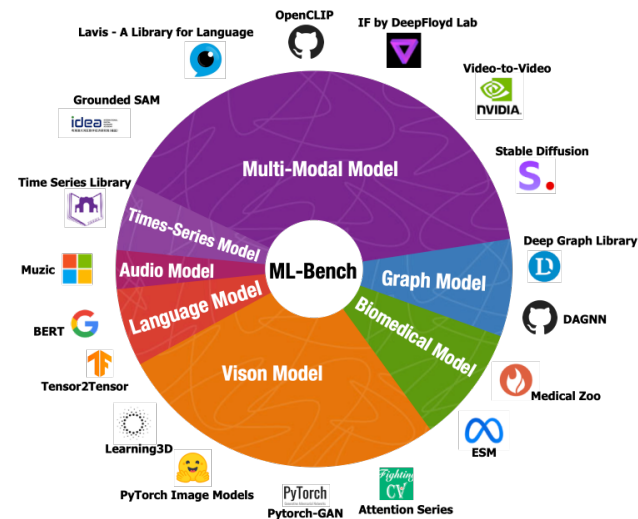


Figure 3: ML-BENCH ENCOMPASSES 18 PROMINENT GITHUB REPOSITORIES AND IT SHOW THE DISTRIBUTION OF 9,641 SAMPLES.

Table 1: Comparison of benchmarks for repository-level code analysis: this comparison focuses on several key attributes across various benchmarks: (1) *Repository Understanding*—the ability to comprehend and navigate the overall structure, dependencies, and functionality of an entire code repository beyond individual files; (2) *Documentation Understanding*—the capability to interpret and utilize documentation elements such as README files to gain insights within the repository; (3) *Cross-File Retrieval*—identifying relevant information across multiple files to complete tasks or resolve issues; (4) *Package Installation*—installing dependencies required for the repository; (5) *Data Downloading*—downloading data required for the task; and (6) *Evaluation*—the methods used to assess and measure the task performance.

Criteria	REPOEVAL [Zhang et al., 2023]	REPOBENCH [Liu et al., 2023]	MLAGENTBENCH [Huang et al., 2024]	SWE-BENCH [Jimenez et al., 2024]	ML-BENCH (OURS)
Repo. Understanding	✓	✓	✗	✓	✓
Doc. Understanding	✗	✓	✓	✗	✓
Cross-File Retrieval	✗	✗	✓	✗	✓
Package Installation	✗	✗	✗	✗	✓
Data Downloading	✗	✗	✗	✗	✓
Evaluation	Similarity / Pass@K	Similarity	Test Accuracy	Success Rate	Pass@K / Success Rate
# of Repositories	14	3,116	/	12	18
# of Tasks	13,710	49,684	13	2,300	9,641

predominantly producing incorrect parameters or referencing non-existent files. Generating bash scripts proved more challenging than generating Python code, pinpointing a capability bottleneck in LLMs. A critical insight from our study is the urgent need for LLMs to comprehend the long code context (the average length is around 150k tokens for the whole repository), not merely to generate code. On the more challenging ML-AGENT-BENCH setup, GPT-4o scores **76.47%** within the OpenDevin agent environment, where agents must configure their environment, navigate code repositories, and effectively generate the necessary code. Similar to Reflexion [Shinn et al., 2023] and CodeAct [Wang et al., 2024], agents iteratively execute actions, refine their approach via feedback, and perform effectively in solving complex tasks. This underscores the potential of self-improvement and incorporating feedback from experience as alternatives to relying solely on instruction tuning with history training data to enhance LLM performance.

To sum up, while recent efforts have explored LLM-based agents for navigating GitHub repositories, such as conducting ML experiments in simplified environments [Huang et al., 2024] or resolving repository issues [Jimenez et al., 2024] (see Table 1), ML-Bench addresses a distinct and critical challenge faced by many researchers: both *setting up* and *executing experiments* using research repositories in-the-wild, where code is often not well-maintained, and experiment settings are not easily configurable, such as:

- Unlike RepoBench [Liu et al., 2023], which focuses on repo-level code auto-completion, ML-Bench requires agents to generate and execute code to run experiments using research repositories. This involves more complex tasks, including environment setup, data preparation, and experiment execution. In our experiment, the models often struggle with this stage (the performance of the strongest model GPT-4o is 32.99%).
- SWE-Bench [Jimenez et al., 2024] tasks agents with locating and modifying specific functions within a file to resolve an issue, providing a pre-deployed testing environment. In contrast, ML-Bench challenges agents to independently configure environments and download necessary data, mimicking real-world research scenarios more closely.
- ML-Bench evaluates the entire workflow of setting up and running ML experiments, from initial repository exploration to result interpretation. This end-to-end approach differentiates it from more narrowly focused benchmarks.

2 ML-BENCH Construction

2.1 Task Formulation and Design Principle

ML-BENCH aims to test LLM performance in real coding scenarios, which includes interpreting repository-level code and writing code to employ existing repositories to meet user demands. In our benchmark, we consider a scenario where a language model, given a GitHub repository $\mathcal{F}$, accesses all files $f \in \mathcal{F}$. Upon receiving an instruction i with arguments $\arg_i \in \mathcal{A}$, the model must generate executable code c that utilizes functions or models from the repository in line with the user instruction i and arguments $\arg_i$.

For ML-BENCH settings, (1) ML-LLM-BENCH provides a complete build environment, allowing us to test output bash scripts generated by LLMs within a Docker environment regarding the correctness and excitability. (2) ML-AGENT-BENCH provides agents with access to an empty Docker environment without essential packages. Agents must attempt to download the requirements for each user

instruction involving the installation of new datasets or Python packages themselves. This design ensures that our testing framework aligns with practical application workflow.

In addition, to address potential data contamination, despite the possibility that code in GitHub repositories may have been exposed to LLMs during pretraining, we mitigate the risk by manually rewriting inputs and outputs and verifying our dataset’s uniqueness against internet searches, ensuring our outputs are free from such contamination.

Table 2: Detailed breakdown of the number of bash script and Python code samples for each repository. The test set contains samples from **14** repositories, while the train set includes **4** additional repositories for the OOD setting. A quarter subset of the test set is also shown. All repository names are hyperlinked for direct access to the corresponding GitHub.

Repository	Train Set		Test Set		1/4 Test Set	
	Scripts	Code	Scripts	Code	Scripts	Code
In-Distribution (ID)						
Video-to-Video (vid2vid)	46	0	13	0	4	0
IF by DeepFloyd Lab (If)	168	175	10	11	4	2
Deep Graph Library (DGL)	553	0	21	0	5	0
Pytorch-GAN (Py-GAN)	1080	0	30	0	8	0
ESM	563	58	15	2	4	1
BERT	962	0	22	0	6	0
OpenCLIP	646	691	10	1	3	0
Lavis - A Library for Language (Lavis)	76	205	4	23	1	6
Time Series Library (TSL)	1449	0	14	0	4	0
Attention Series (EAP)	95	5	24	0	5	0
Out-Of-Distribution (OOD)						
Grounded-SAM	/	/	12	8	2	3
PyTorch Image Models (Py-IM)	/	/	5	0	1	0
muzic	/	/	17	1	4	1
Learning3D	/	/	17	0	4	0
Stable Diffusion (SD)	2253	0	/	/	/	/
Medical Zoo (MedZooPy)	490	0	/	/	/	/
Time Series Library (TCL)	196	0	/	/	/	/
Tensor2Tensor	0	248	/	/	/	/
Total	8577	736	214	46	55	13

2.2 Summary of Data

We construct ML-BENCH from 18 diverse repositories, each reflecting varying complexity and tasks, while filtering out substandard samples. The data quantities and breakdown per repository are detailed in Table 2. Regarding the code language, our annotated output includes both **bash scripts**, which invoke Python files with specific arguments, and **Python code**, which calls functions directly from the repository. Bash scripts significantly outnumbered Python code snippets.

On average, each repository contributed approximately 480 examples, summing up to 9,641 examples. For our experiments involving the fine-tuning of open-source LLMs, we split the dataset based on code origin: The **In-Distribution (ID)** approach utilizes data from the same repository both for training and testing, allowing repository-specific code to be exposed to models during fine-tuning. In contrast, the **Out-Of-Distribution (OOD)** method employs disjoint sets for training and testing, encompassing eight repositories—half for model training and the remaining for evaluation. The overall statistics and further detailed data metrics for each repository utilized can be found in Appendix B.1.

2.3 Data Collection and Annotation Pipeline

Eight computer science graduate students with proficient programming abilities contributed to the data annotation of ML-BENCH, with each repository’s related data being the responsibility of one annotator and an additional reviewer ensuring data accuracy. Annotators were permitted to use GPT-4 to expedite the annotation process, although manual verification and adjustments were required. Annotating a repository took approximately 5-10 hours. The annotation workflow is shown in Figure 4:

(1) **README file Selection:** Annotators commenced by meticulously reviewing repository contents to identify all README files, including those within various subdirectories, each covering different functionalities. On average, a GitHub repository included 12 README pages, with one notable repository, DGL, comprising 154 README files. (2) **Task Mining:** Annotators identify practical tasks from README files, along with corresponding code examples, averaging nine tasks per repository, thus capturing the representative functionality of each GitHub repository. (3) **Arguments**

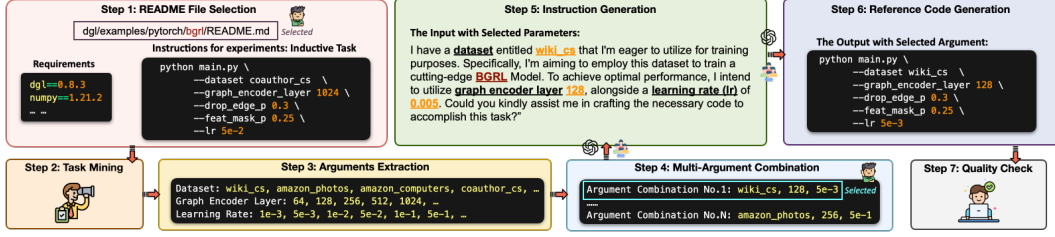


Figure 4: The construction pipeline of our ML-BENCH.

Extraction: Beyond task identification, annotators with machine learning expertise extracted key parameters essential for task completion, targeting representative parameters commonly employed in practical experiments. **(4) Multi-Argument Combination:** With tasks and arguments identified, annotators create diverse combinations of argument values, essential for constructing scenarios that represent real-world applications of repository code. **(5) Instruction Generation:** Utilizing ChatGPT, we generate task-specific instructions for each argument set, applying templates to ensure diversity and explicit argument inclusion, detailed in Appendix B.2. **(6) Reference Code Generation:** For each argument combination, we develop code templates to generate accurate ground truth code for the targeted tasks. **(7) Quality Check:** The dataset underwent stringent quality checks, particularly for code executability and argument accuracy, with any non-compliant data being revised or discarded. This ensures that the instructions precisely align with user requirements, thereby upholding the integrity and applicability of the ML-BENCH benchmark.

3 ML-LLM-BENCH Experiments

3.1 ML-LLM-BENCH Setup

Our experimental inputs include human instructions and the entire repository code (including README files). We present three distinct experimental setups to evaluate the models. Given that current models cannot process the entire code context, the three scenarios range from ideal to extreme. **Oracle Segment (Oracle):** For the Oracle setup, annotators identify and record crucial segments within README files — referred to as "Oracle Segments" — that contain necessary codes and textual explanations pivotal for completing the prescribed tasks. These segments serve as the foundational source to derive the ground truth code, ensuring that models can access all critical evidence when generating code. **BM25 Retrieval (Retrieval):** In this setup, we employ a BM25 retriever to extract segments from the repository’s documentation, including README files, that are relevant to the given instructions. This method aims to mimic a more automated way of narrowing down necessary information without human pre-selection. **Code (Code):** This setting exposes the model to the entire code repository. All files within the repository, with README files placed at the forefront, are presented as input to the model. Due to model context limitations, texts are truncated when necessary, and potential information loss is analyzed and documented in Appendix B.1. Please refer to the supplementary materials for further details on implementing the BM25 retriever.

3.2 Evaluation Metrics

The generated code must be executable and adhere to the parameters outlined in the user instructions. We use Pass@K as our metric for evaluation, with K representing the number of generation attempts allowed. Pass@K measures the likelihood of the model producing at least one correct code execution in those K tries (given unit tests).

3.3 Experimental Results

As presented in Table 3, we conducted evaluations on a set of LLMs including GPT-4o (model name: gpt-4o-2024-05-13), GPT-4 (model name: gpt-4-1106-preview), GPT-3.5 (model name: gpt-3.5-turbo-16k-0613), and the Claude 3 model family (Claude-3-Opus, Claude-3-Sonnet, Claude-3-Haiku). Moreover, we selected CodeLlama-7b-Instruct, DeepSeek-Coder-6.7b-Instruct, and Llama-2-7b-chat-hf (LLaMA 2) to explore the effects of fine-tuning with an 8k token length limit with 4 A100s. The findings suggest that while GPT-4o exhibited the highest scores across the test cases, the untrained models, such as LLaMA-2-7b, performed poorly on the ML-LLM-BENCH, even after in-distribution (ID) fine-tuning. Fine-tuning on out-of-distribution (OOD) data indicated that models could benefit from training on similar tasks, though not necessarily from the same repository. Moreover, the

Table 3: Pass@1/5 scores for models on the SCRIPTS|CODE (bash script, Python code) partition of ML-LLM-BENCH. †denotes instruction-tuned models. Results are shown for the **Oracle**, **Code**, and **Retrieval** settings. Results under ID and out-of-distribution (OOD) are reported after instruction fine-tuning.

Models	Oracle ^{SCRIPTS CODE}		Code ^{SCRIPTS CODE}		Retrieval ^{SCRIPTS CODE}	
	Pass@1	Pass@5	Pass@1	Pass@5	Pass@1	Pass@5
Human	/	/	86.76	/	/	/
<i>Close-Source LLMs</i>						
GPT-4o	36.42 ^{31.37 56.83}	50.13 ^{44.26 78.89}	32.99 ^{31.44 39.87}	46.20 ^{43.58 61.54}	26.16 ^{19.47 55.52}	30.44 ^{24.73 76.92}
GPT-4	33.82 ^{29.09 53.85}	48.53 ^{41.81 76.92}	30.88 ^{29.09 38.46}	45.59 ^{41.82 61.54}	22.06 ^{14.55 53.85}	27.94 ^{16.36 76.92}
GPT-3.5	27.94 ^{21.81 53.85}	38.23 ^{30.91 69.23}	15.07 ^{0.09 38.46}	30.14 ^{23.64 53.85}	13.70 ^{5.45 46.15}	24.66 ^{14.55 69.23}
Claude-3-Opus	25.52 ^{12.15 67.39}	36.92 ^{27.57 80.43}	13.46 ^{0.70 43.48}	35.39 ^{30.37 58.70}	10.00 ^{3.27 41.30}	22.69 ^{11.22 76.09}
Claude-3-Sonnet	21.92 ^{18.18 38.46}	34.25 ^{27.27 61.54}	27.40 ^{25.45 30.76}	35.62 ^{30.91 53.85}	9.59 ^{3.64 38.46}	20.55 ^{9.09 69.23}
Claude-3-Haiku	18.46 ^{11.68 50.00}	30.38 ^{20.09 78.26}	25.38 ^{22.90 36.96}	32.31 ^{28.04 52.17}	8.08 ^{3.74 28.26}	16.92 ^{7.48 60.87}
<i>Open-Source LLMs</i>						
CodeLlama-7b	8.85 ^{3.37 32.60}	21.15 ^{11.68 65.22}	1.54 ^{0.47 6.52}	8.85 ^{2.80 36.96}	0.77 ^{0.00 4.34}	8.85 ^{2.80 36.96}
DeepseekCoder-6.7b	9.23 ^{0.46 30.43}	24.23 ^{14.02 71.74}	3.85 ^{1.89 13.04}	10.38 ^{6.07 30.43}	5.00 ^{3.27 13.04}	14.23 ^{9.81 34.78}
LLaMA-2-7b	2.27 ^{0.13 5.70}	4.77 ^{2.47 6.22}	0.00	0.00	0.00	0.00
<i>Finetuned LLMs w/ the Out-Of-Distribution (OOD)</i>						
CodeLlama-7b †	15.76 ^{12.14 32.61}	28.46 ^{19.62 69.57}	/	/	1.92 ^{0.47 8.70}	5.38 ^{1.40 23.91}
DeepseekCoder-6.7b †	16.15 ^{14.95 34.78}	31.15 ^{24.30 58.70}	/	/	10.38 ^{6.54 28.26}	26.15 ^{17.29 67.39}
LLaMA-2-7b †	5.31 ^{2.47 10.86}	6.03 ^{3.12 11.64}	/	/	2.77 ^{1.30 5.34}	5.31 ^{2.47 10.86}
<i>Finetuned LLMs w/ the In-Distribution (ID)</i>						
CodeLlama-7b †	17.69 ^{15.42 28.26}	30.77 ^{21.96 71.74}	/	/	2.69 ^{0.47 13.04}	9.62 ^{3.27 39.13}
DeepseekCoder-6.7b †	21.92 ^{12.16 65.22}	30.77 ^{20.56 78.26}	/	/	2.69 ^{1.40 8.70}	10.00 ^{5.61 30.43}
LLaMA-2-7b †	6.54 ^{2.33 26.09}	8.38 ^{4.45 32.17}	/	/	1.15 ^{0.00 6.52}	3.08 ^{4.67 15.22}

Table 4: Agent evaluation results on the ML-AGENT-BENCH. The success rate, number of solved instances, and the average cost per solved instance are reported for each agent and language model combination. † Evaluation is conducted on a quarter subset of the test set due to budget constraints. § OpenDevin is available on <https://github.com/OpenDevin/OpenDevin>.

Agent	Model Name	Success Rate [†] (%)	# of Solved Instances	\$ Avg. Cost
AutoGen [Wu et al., 2023] SWE-Agent [Yang et al., 2024] Aider [Gauthier]	gpt-4-1106-preview	8.82	6	1.28
	gpt-4-1106-preview	42.64	29	1.91
	gpt-4o	64.38	47	-
OpenDevin [§]	gpt-4o-2024-05-13	76.47	51	0.25
	gpt-4-1106-preview	58.82	40	1.22
	gpt-3.5-turbo-16k-0613	13.23	9	0.12

performances on ID data implied that even after task-relevant fine-tuning, the results from 7B-scale open-source models could not outperform the closed-source counterparts. The oracle setting outcomes demonstrate that providing models with the correct reference solutions is effective for task completion. A retrieval approach not specifically designed for the task might lead to suboptimal results, potentially hindering performance.

4 ML-AGENT-BENCH Experiments

4.1 ML-AGENT-BENCH Setup

In ML-AGENT-BENCH, as shown in Figure 5, we provision a sandbox environment as the testing ground for agents. The sandbox offers a fundamental setup, such as a configurable Docker image, allowing agents to modify and execute commands freely within the simulation. Agents are granted the ability to execute bash scripts or interact with IPython notebooks. The agents must interact with this environment, perusing code within repositories—regardless of the extended code or required parameters—to accumulate comprehensive information. This process necessitates successive actions, with the agent autonomously determining the correctness of each step and iteratively refining its approach upon encountering errors. We expect the agents’ outputs to differ from previous LLM settings due to the dynamic and interactive nature of the tasks.

Taking cues from projects such as CodeAct [Wang et al., 2024], recent agent frameworks, including SWE-Agent [Yang et al., 2024], Aider [Gauthier], OpenDevin (<https://github.com/OpenDevin/OpenDevin>), provide a well-defined suite of impactful ac-

tions that bridge the agent with its operational environment. These agents are designed to mimic the workflow of human programmers, thoroughly parsing and employing a repository. To facilitate this, the agents are endowed with the ability to execute *any* Python code and bash commands within a secure and isolated Linux OS sandbox, providing an ideal setting for our benchmark evaluations. In each instance, ML-AGENT-BENCH initiates an isolated docker container sandbox where all agents’ bash commands are executed, with the outcomes returned as observations. Different agent frameworks implement environmental interactions in varying ways, with each action yielding observations for AI agents. Here ML-AGENT-BENCH is essentially assessing the effectiveness of different environments. In ML-AGENT-BENCH, a configurable workspace directory contains repositories agents are to handle, installed within this safe sandbox, providing access for the agents to interact with and process as needed. For evaluation, instead of using the Pass@K metric in the ML-LLM-BENCH setting, we are emphasizing the agent’s effectiveness in fulfilling user requirements through interactive execution rather than predetermined outputs (**Success Rate**). Success is characterized not just by code generation but also by the agent’s ability to alter or create new files, such as a *run.sh*, to achieve the execution required by the instruction.

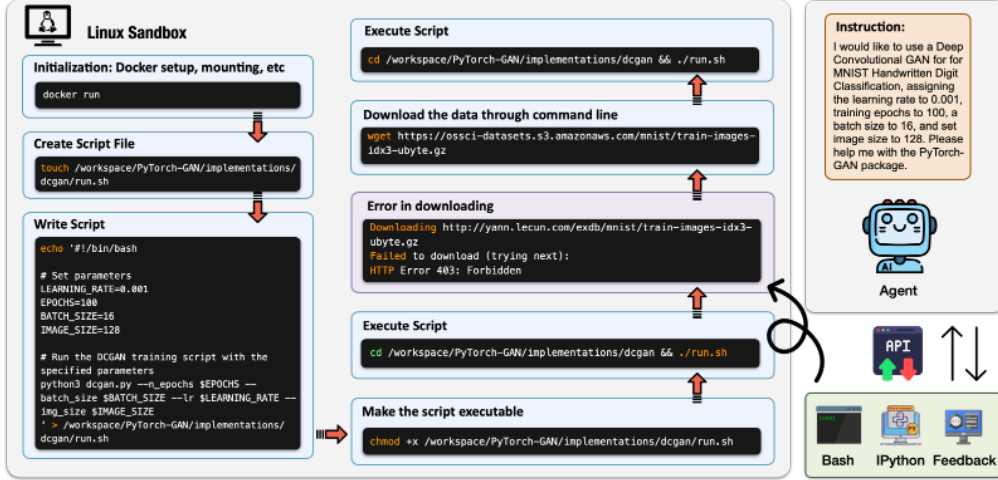


Figure 5: An example of the ML-AGENT-BENCH evaluation runtime

4.2 Experimental Results

In Table 4, we detail the performance of various agents such as AutoGen, SWE-Agent, and Aider, as well as OpenDevin equipped with diverse GPT language models, evaluated on a quarter subset of the test set. OpenDevin, utilizing GPT-4o (model name: gpt-4o-2024-05-13), achieved the best results, striking an excellent balance between cost and performance. The success rate, the number of instances successfully solved, and the average cost per solved instance were the critical metrics for this evaluation. As demonstrated by the varied performance of agents using the identical *gpt-4-1106* model, it becomes evident that the choice of agent framework significantly impacts the effectiveness of an agent. This discrepancy in success rates and average costs accentuates the potential for future advancements in agent architecture to enhance performance further.

5 Analysis

5.1 Data Leakage

Our early experiments found that models tended to generate Python code instead of bash script regardless of whether the input is script or code. This is related to the fact that model trainers expect they can get high scores on other code benchmarks, causing these polished repositories to be more likely to be put into the training set compared to lesser-known ones, and code data is a larger percentage than script data.

To mitigate the influence of data leakage, the type and parameters of the generated result are checked to see whether they are the same as those present in the provided documents and user instructions before execution. We show the updating status for all repositories and the cutoff date of training data for models in Appendix B.1.

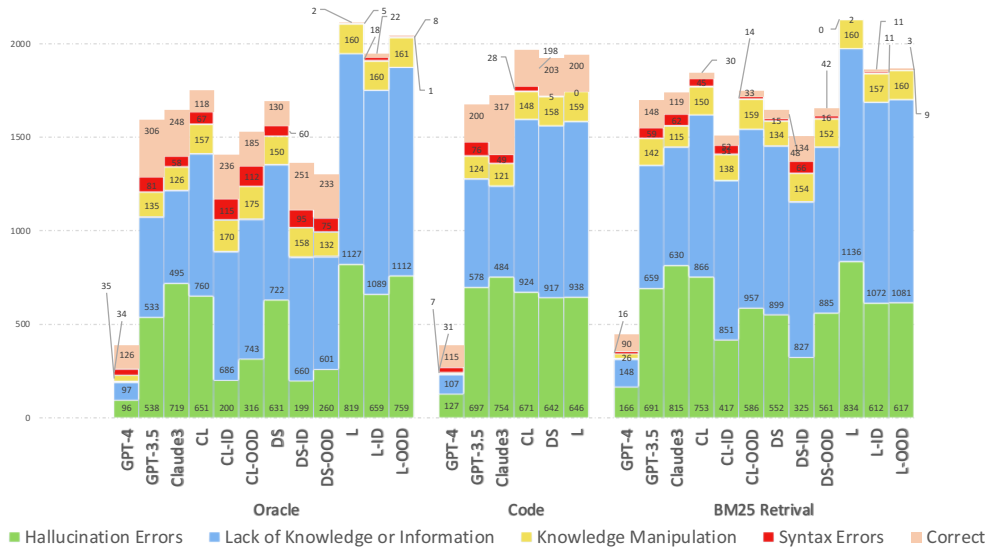


Figure 6: Quantification of models and settings errors with five attempts. The total statistic results are 1300 for the full test set. Statistical results that exceed these numbers are caused by multiple errors made on one result simultaneously. For models, CL means CodeLlama, DS means deepseek-coder, and L means LLaMA. Raw means that the model is not fine-tuned. ID means that the model is fine-tuned in an in-distribution setting. OOD means that the models are fine-tuned in an out-of-distribution setting. Here, Claude3 means Claude-3-Haiku.

5.2 Error Analysis

By analyzing the execution log, we find that the errors for models and agents in ML-BENCH fall into five categories:

Hallucination Errors (E1): These errors include instances when the models misinterpreted the user’s intention, misplaced Python code and bash script, or generated random or irrelevant code.

Lack of Knowledge or Information (E2): This type of error primarily stems from the model’s inability to fulfill user requirements based on crucial parameters. Possible types of errors are as follows:

1. Code inner information. The models sometimes lack sufficient information necessary to satisfy the user’s requirements. For instance, this deficiency might involve missing parameter names (`--lr` and `--learning-rate`) or unavailable options (it only accepts 'CiteSeer' when the input given was 'citeseer').
2. Domain knowledge. The models sometimes lack the domain-specific knowledge required to handle certain instances. For example, in BERT, the models simultaneously generated `--case=True` and `--model=uncased`.
3. Grammar knowledge. This happens when the models incorrectly identify and handle certain command line symbols. Like the \$ symbol, which could affect execution.
4. Local data information. The models were not supplied with enough crucial parameter information for specific commands, leading to the inability to find paths and successful execution. While less common, this error was particularly notable in the OpenCLIP.

Knowledge Manipulation (E3): Take BERT, for instance, where the model needed to integrate `DIR=/model/` and `--model_dir=$DIR` to form `--model_dir=/model`. There were also cases where it couldn’t merge `/model_name` in `/data` into a proper path format like `/data/model_name`. Similar incidents were observed in OpenCLIP.

Syntax Errors (E4): These errors cover instances of incorrect code generation of syntax errors instead of hallucination, mainly Python syntax errors such as the use of undefined variables. These errors arise from cases that prefer generating Code.

Operational Error (E5 – ML-AGENT-BENCH Only): These errors only occur in ML-AGENT-BENCH. Less powerful agents, such as those using base models like GPT-3.5, may have difficulty understanding the task in the execution sandbox. In these cases, the agent gives up easily or exits without finishing the tasks. Interestingly, these errors are less observed in advanced model settings.

On ML-LLM-BENCH, we find that Claude 3 has more hallucinations than GPTs for closed-source models. However, its ability to fulfill requirements (represented by E2) is better than that of GPT-3.5. Under the retrieval setting, neither GPT-3.5 nor Claude 3 exhibits an increase in hallucination but an increase in the proportion of E2 compared to the Code setting. For GPT-4, both E1 and E2 increased because GPT-4 generates code without considering the content in cases involving task-irrelevant information, except for GPT-3.5 and Claude 3.

Compared to the Code setting, Oracle provision decreases the quantities of E1 and E2, while the differences in E3 and E4 are insignificant. This suggests that whether an Oracle is provided does not significantly affect the closed-source models’ knowledge manipulation and Python code generation abilities. We tend to attribute these to the models’ inherent ability rather than the reference.

The ML-AGENT-BENCH showcases potential in incorporating feedback from the experience, leading to fewer E1 and E4 errors. Yet, compared to ML-LLM-BENCH, ML-AGENT-BENCH is more prone to E2 and E3 errors due to file type discrepancies from the set output type. Because of the great flexibility of the sandbox environment, we observe increased hallucinations with each step, including installing conflicting dependencies and navigating incorrect or even illegal directories.

Meanwhile, E5 happens in less powerful agents. The agent often ignores the instruction to export the final solution to `run.sh`, or refuses to answer the question immediately after entering the virtual sandbox. Compared to GPT-4, GPT-4o shows enhanced abilities in sourcing relevant information from repositories, leading to markedly fewer errors associated with E2 and E3. Yet, it exhibits a tendency toward more frequent hallucinations than GPT-4, for instance, installing incorrect packages.

For a detailed error analysis and quantitative performance visualization on both setups, refer to Appendix B.1.

6 Conclusion

Our study evaluates the performance of Large Language Models (LLMs) like GPT-4 in repository-scale code understanding and autonomous code interaction. Our evaluation includes two setups: ML-LLM-BENCH for text-to-code conversion and ML-AGENT-BENCH for end-to-end task execution. Results show GPT-4 achieving a Pass@5 rate over 50% in ML-LLM-BENCH and a 76.47% success rate in ML-AGENT-BENCH, highlighting areas for improvement in handling hallucinations and bash script generation.

Limitation

Our study, while comprehensive within its scope, is subject to certain limitations that stem primarily from linguistic and data source constraints.

Linguistic Limitation - English as a Working Language We exclusively focused on English for our analyses and model development. This choice, while pragmatic due to English’s prevalence in scientific literature and technical documentation, inherently limits the generalizability of our findings. English, as a language, possesses unique syntactic and semantic structures that may not be representative of other languages. Consequently, the applicability of our results to non-English contexts is uncertain. This linguistic limitation also restricts the diversity of perspectives and cultural nuances that non-English documents could offer.

Data Source Limitation - Reliance on GitHub Repositories in English Our reliance on GitHub repositories with documents exclusively in English introduces a selection bias. GitHub, while rich in open-source projects and documentation, may not comprehensively represent the broader landscape of software development practices and trends globally. This choice potentially overlooks significant contributions and insights from non-English-speaking communities. This limitation might impact the development of tools and models tailored to a more diverse set of programming environments and community needs.

Methodological Limitation - Relying on Pre-built Machine Learning Packages In our methodology, we utilized existing machine learning packages instead of developing algorithms from scratch. While this approach allowed us to leverage well-established, tested, and optimized tools, it also introduces certain constraints. Dependence on pre-built packages means our work is confined to the capabilities and limitations of these tools. This reliance could limit our ability to fully explore novel or unconventional approaches possible with custom-built algorithms. Moreover, this choice potentially impacts the reproducibility and customization of our findings. Researchers who seek to build upon our work may encounter similar constraints imposed by the pre-built packages we utilize. These limitations can hinder innovation and adaptation in different contexts or for specific usage.

Scope Limitation - Tasks Limited to README File Descriptions By strictly adhering to the specified tasks, our study may overlook potential applications or challenges not explicitly documented in README. This limitation can result in a narrower understanding of the tools we examined, as it fails to explore their full potential and applicability. The reliance on README descriptions also assumes that these documents comprehensively and accurately reflect all relevant aspects of the repositories, which may not always be accurate. Important tasks or nuances might be undocumented or underrepresented in these files.

Ethics Statement

In our work, we have carefully considered the ethical implications of our work, particularly in data annotation and related activities. Our methodologies and processes have been meticulously designed to ensure they are free from moral concerns. We affirm that our research practices, including data handling, have been conducted with the utmost integrity and in compliance with ethical standards.

Our approach has been guided by principles prioritizing respect for data integrity, transparency in our methods, and adherence to established ethical guidelines.

References

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiushi Du, Zhe Fu, et al. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024.
- Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*, 2022.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Fenia Christopoulou, Gerasimos Lampouras, Milan Gritta, Guchun Zhang, Yinpeng Guo, Zhongqi Li, Qi Zhang, Meng Xiao, Bo Shen, Lin Li, et al. PanGu-Coder: Program synthesis with function-level language modeling. *arXiv preprint arXiv:2207.11280*, 2022.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. CodeBERT: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*, 2020.
- Paul Gauthier. How aider scored sota 26.3 <https://aider.chat/2024/05/22/swe-bench-lite.html>. Accessed: 2024-06-05.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021a.
- Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021b.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation. In *Forty-first International Conference on Machine Learning*, 2024.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>.
- Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*, pages 18319–18345. PMLR, 2023.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023.

- Gabriel Orlanski, Kefan Xiao, Xavier Garcia, Jeffrey Hui, Joshua Howland, Jonathan Malmaud, Jacob Austin, Rishabh Singh, and Michele Catasta. Measuring the impact of programming language distribution. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Xiangru Tang, Bill Qian, Rick Gao, Jiakang Chen, Xinyun Chen, and Mark B. Gerstein. BioCoder: A benchmark for bioinformatics code generation with contextual pragmatic knowledge. *arXiv preprint arXiv:2308.16458*, 2023a.
- Xiangru Tang, Yiming Zong, Yilun Zhao, Arman Cohan, and Mark Gerstein. Struc-Bench: Are large language models really good at generating complex structured data? *arXiv preprint arXiv:2309.08963*, 2023b.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents, 2024.
- Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D.Q. Bui, Junnan Li, and Steven C. H. Hoi. CodeT5+: Open code large language models for code understanding and generation. *arXiv preprint arXiv:2305.07922*, 2023.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-Agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. *arXiv preprint arXiv:2303.12570*, 2023.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Zihan Wang, Lei Shen, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and Jie Tang. CodeGeeX: A pre-trained model for code generation with multilingual evaluations on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, page 5673–5684, New York, NY, USA, 2023.
- Wangchunshu Zhou, Yuchen Jiang, Long Li, Jialong Wu, Tiannan Wang, Shi Qiu, Jintian Zhang, Jing Chen, Ruipu Wu, Shuai Wang, Shiding Zhu, Jiyu Chen, Wentao Zhang, Ningyu Zhang, Huajun Chen, Peng Cui, and Mrinmaya Sachan. Agents: An open-source framework for autonomous language agents. *arXiv preprint arXiv:2309.07870*, 2023.

A Related work

A.1 Code Generation

Code generation in natural language processing (NLP) has been a significant research topic, leading to the development of various methodologies and benchmarks, as seen in [Cassano et al., 2022, Chen et al., 2021, Christopoulou et al., 2022, Li et al., 2022, Orlanski et al., 2023, Tang et al., 2023a,b, Wang et al., 2023]. Current benchmarks primarily aim to enhance function-level code generation capabilities. However, ML-BENCH diverges by integrating code generation to streamline the usage of repositories within real-world workflows. For a comparative overview, see Table 5. The goal of function-level code generation is the creation of code snippets tailored to user needs or to augment code completion processes [Feng et al., 2020, Li et al., 2022], which includes the development of code LLMs [Bi et al., 2024, Zheng et al., 2023].

Table 5: Comparison of benchmarks: characterizing existing function-level benchmarks and ML-BENCH.

Bench Name	Type	Language	# Samples
ML-Bench	Task Execution	Python & Bash	9,641
HumanEval [Chen et al., 2021]	Function Completion	Python	164
MBPP [Austin et al., 2021]	Function Completion	Python	1,000
DS-1000 [Lai et al., 2023]	Function Completion	Python	1,000
APPS [Hendrycks et al., 2021a]	Function Completion	Python	10,000

A.2 Agent

The integration of AI agents in software development is rapidly advancing, with projects like OpenDevin², SWE-agent [Yang et al., 2024], AutoGen [Wu et al., 2023], and Aider [Gauthier] showcasing diverse methodologies for augmenting developers’ capabilities. OpenDevin[§] leverages open-source technologies to automate coding and debugging, thereby streamlining development workflows. SWE-agent’s ACI allows language models to independently tackle software engineering tasks, exhibiting impressive outcomes in benchmark tests. AutoGen’s collaborative agent framework melds conversational AI with human and digital tools to automate a breadth of tasks, from programming to problem-solving. Finally, Aider brings LLMs directly into the coding process, enabling true co-editing experiences between AI models like GPT-4o, Claude 3 Opus, and developers within git repositories, enhancing code editing and project management.

²<https://github.com/OpenDevin/OpenDevin>.

B Dataset Details

B.1 Details of Selected GitHub Repositories

As depicted in Table 6, our selection encompasses a range of GitHub repositories varying from language and graph models to multimodal and time-series models. Each repository is chosen for its high-quality contributions to the field and its popularity among the development community, indicated by the number of stars. The repositories, diverse in their updates and number of README files, provide a snapshot of the current landscape of models available on GitHub.

Table 6: Comprehensive information on selected GitHub repositories. The column labeled "#README" refers to the number of README files contained within each listed GitHub repository.

Domain	GitHub	Stars	URL	#README	Last Updated
Language Model	BERT	35693	https://github.com/google-research/bert	1	2020.03.11
	Tensor2Tensor	14280	https://github.com/tensorflow/tensor2tensor	9	2023.04.01
Graph Model	DGL	12429	https://github.com/dmlc/dgl	154	2023.11.16
Biomedical Model	ESM	2462	https://github.com/facebookresearch/esm	8	2023.06.27
	MedicalZooPytorch	1516	https://github.com/black0017/MedicalZooPytorch	21	2022.02.07
Vision Model	PyTorch-GAN	14947	https://github.com/eriklindernoren/PyTorch-GAN	1	2021.01.07
	Learning3d	579	https://github.com/vinit5/learning3d	1	2023.10.24
	External-Attention-pytorch	9949	https://github.com/xmu-xiaoma666/External-Attention-pytorch	1	2023.10.25
	Pytorch-image-models	30400	https://github.com/huggingface/pytorch-image-models	1	2023.11.09
Audio Model	Muzic	3866	https://github.com/microsoft/muzic	8	2023.12.06
Multi-Modality	LAVIS	7300	https://github.com/salesforce/lavis	8	2023.09.25
	IF	7237	https://github.com/deep-floyd/if	1	2023.06.03
	OPEN-CLIP	6856	https://github.com/mlfoundations/open_clip	1	2023.11.01
	Stable Diffusion	31506	https://github.com/Stability-AI/stablediffusion	1	2023.03.25
Video	Segment-Anything	11976	https://github.com/IDEA-Research/Grounded-Segment-Anything	3	2023.12.11
	Vid2Vid	8393	https://github.com/NVIDIA/vid2vid	2	2019.07.04
Time-Series Model	Time-Series-Library	2670	https://github.com/thuml/Time-Series-Library	1	2023.11.10

B.2 Templates for Diverse Instruction Construction

To generate precise instructions, we have developed a range of instruction templates that serve as the foundation for the tasks, ensuring that all necessary arguments are present and accounted for. For a detailed look, please refer to the instruction templates outlined in Table 7. These templates are instrumental in guiding the generation process, embodying diversity and adaptability while maintaining the integrity of the input information. The templates in the table guide users to regenerate similar instructions with varied expressions while keeping the task requirements intact, promoting diversity in instructional outputs for the model to process and execute.

Table 7: Examples of constructed instructions for diverse task templates. "ESMFold" in the context of the table refers to a task related to protein structure prediction using a particular machine learning model or function. The `esmfold_v1()` and `esm1v_t33_650M_UR90S_1()` mentioned in the instructional templates are names of functions or models within a software package designed to predict the three-dimensional structure of proteins based on their amino acid sequence. In the examples provided in the table, the "ESMFold Structure Prediction" task requires inputs such as a sequence of amino acids and specific model parameters, including learning rates and output file paths. Users are asking for assistance in writing code that utilizes the ESMFold model to predict protein structures and save the results to a specified output file in PDB format, which is a common format for representing 3D structures of large biological molecules.

Templates of Instruction Construction
You need to use your creativity and rewrite each query three times. You need to make sure that all the required arguments are provided. Example 1 Input: I have a data sequence MKTVRQERLKSI, and I want to use <code>esm1v_t33_650M_UR90S_1()</code> to finish ESMFold Structure Prediction task, set learning rate to 0.000001 then output to output.pdb. Can you help me write some code? Require arguments: sequence: MKTVRQERLKSI model: <code>esm1v_t33_650M_UR90S_1()</code> task: ESMFold Structure Prediction task learning rate: 0.000001 output path: output.pdb Output: 1. Input data: MKTVRQERLKSI - model: <code>esm1v_t33_650M_UR90S_1()</code> - task: ESMFold Structure Prediction - output path: output.pdb - learning rate: 0.000001 Help me write example code. 2. I possess a data sequence with the characters MKTVRQERLKSI, and I'm interested in leveraging the <code>esm1v_t33_650M_UR90S_1()</code> function to accomplish the ESMFold Structure Prediction task. Please set the learning rate to 0.000001. Subsequently, I'd like to generate an output file named output.pdb. Would you be able to assist me by providing the necessary code? 3. I possess a fascinating data sequence bearing the distinctive characters MKTVRQERLKSI. I intend to harness the power of the <code>esm1v_t33_650M_UR90S_1()</code> function, a formidable tool indeed, to conquer the challenging task of ESMFold Structure Prediction. Please set the learning rate to 0.000001. Once victorious, I aim to put the outcome into a file known as output.pdb. Would you be willing to embark on this coding odyssey with me and craft the necessary script? Example2 Input: I want to use <code>esmfold_v1()</code> to finish the ESMFold Structure Prediction task, set the learning rate to 0.04 and batch size to 5, then output to ./output/data.pdb. Can you help me write some code? Require arguments: model: <code>esmfold_v1()</code> task: ESMFold Structure Prediction task learning rate: 0.04 batch size: 5 Output: 1. Do you know ESM? I need to do the ESMFold structure prediction task. I want to use model <code>esmfold_v1()</code> and set the learning rate to 0.04 and batch size to 5. Can you help me? 2. I have a task at hand involving ESMFold Structure Prediction, and I'm interested in harnessing the <code>esmfold_v1()</code> model for this purpose. I want to set the learning rate to 0.04 and the batch size to 5. Could you guide me through the process of writing the requisite code? 3. I'm looking to employ the <code>esmfold_v1()</code> function to complete the ESMFold Structure Prediction task, and I would like to set the learning rate to 0.04 and batch size to 5. Could you assist me in crafting the necessary code? You must follow the examples, keep the information unchanged, and write some new instructions. Your output should be diverse. You are encouraged to write long instructions. The instructions should be very different. Input: I'm interested in utilizing <code>./model/cased_L-12_H-768_A-12</code> for fine-tuning the SQUAD2.0 task. I plan to establish a train batch size of 128. Could you assist me in writing the necessary code? Require arguments: model path: <code>./model/cased_L-12_H-768_A-12</code> task: SQUAD2.0 batch size: 128 Output: 1. 2. 3.

B.3 Token Length Analysis

For a detailed examination of the token lengths encountered during our model evaluations, we present two tables (Table 8 and 9) comparing the token counts across a variety of experimental settings. The measurements were made using the CodeLlama tokenizer and GPT-3.5 tokenizer.

Table 8: Average input token length of instructions for CodeLlama and GPT-3.5* in various experimental settings. *We use tiktoken (<https://github.com/openai/tiktoken>) to calculate the token number for GPT-3.5.

Repository	CodeLlama						GPT 3.5					
	Code		Retrieval		Oracle		Code		Retrieval		Oracle	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
In-Distribution (ID)												
DGL	5,466,687	5,466,687	312	2,603	179	138	4,455,349	4,455,349	275	2,011	143	110
BERT	138,445	138,445	401	344	372	375	112,104	112,104	335	280	287	290
ESM	27,107,031	27,107,031	585	438	177	173	22,227,765	22,227,765	486	273	139	136
Py-GAN	146,570	146,570	532	897	314	314	119,454	119,454	433	744	268	268
Lavis	16,827,026	16,827,026	471	401	1984	1984	13,714,026	13,714,026	372	325	1547	1547
External-Attention (EAP)	449,381	449,381	1155	526	105	118	346,898	346,898	857	412	69	80
If	68,316	68,316	1390	1,642	3023	3023	55,677	55,677	1119	1,330	2367	2367
vid2vid	146,696	146,696	408	1615	556	565	111,783	111,783	338	481	416	416
OpenCLIP	6,143,829	6,143,829	415	491	5420	5420	5,037,939	5,037,939	350	405	4397	4397
TSL	337,114	337,114	382	902	345	345	273,062	273,062	315	731	276	276
Out-Of-Distribution (OOD)												
Grounded-SAM	/	16,726,416	/	898	/	164	/	13,715,662	/	754	/	113
Py-IM	/	5,608,249	/	8,025	/	89	/	4,542,681	/	6,415	/	68
muzic	/	13,325,828	/	616	/	83	/	10,860,549	/	507	/	64
Learning3D	/	320,157	/	640	/	50	/	256,110	/	596	/	45
SD	258,096	/	501	/	234	/	209,058	/	412	/	183	/
MedZooPy	2,701,443	/	1,302	/	133	/	2,150,168	/	1,101	/	99	/
TCL	18,696,614	/	345	/	116	/	15,114,250	/	291	/	96	/
Tensor2Tensor	4,598,727	/	501	/	192	/	3,678,980	/	432	/	153	/

Table 9: Average output token length of code for GPT-3.5* and CodeLlama to generate across different datasets (Train Set, Test Set, 1/4 Test Set) for various repositories, separated by Python Code and Bash Script. *We use tiktoken (<https://github.com/openai/tiktoken>) to calculate the token number for GPT-3.5.

Repository	Train Set				Test Set				1/4 Test Set			
	GPT-3.5		CodeLlama		GPT-3.5		CodeLlama		GPT-3.5		CodeLlama	
	Python	Bash	Python	Bash	Python	Bash	Python	Bash	Python	Bash	Python	Bash
In-Distribution (ID)												
DGL	/	21.15	/	28.05	/	18.24	/	24.33	/	21.60	/	28.40
BERT	/	121.98	/	181.60	/	120.14	/	179.36	/	127.67	/	189.50
ESM	142.79	37.80	183.84	52.44	127.50	37.47	167.50	52.40	127.00	40.00	167.00	54.25
Py-GAN	/	28.63	/	43.25	/	27.30	/	41.10	/	27.00	/	40.88
Lavis	222.95	36.05	313.97	51.72	211.30	34.75	300.57	49.25	187.33	37.00	267.00	51.00
EAP	170.87	/	239.68	/	121.63	/	174.96	/	146.20	/	205.60	/
If	243.47	160.00	325.42	201.00	272.19	/	362.57	/	269.33	/	361.83	/
vid2vid	/	85.65	/	112.67	/	79.85	/	104.85	/	63.25	/	84.75
OpenCLIP	859.31	/	1236.63	/	839.55	/	1207.91	/	913.33	/	1313.33	/
TSL	/	152.98	/	205.82	/	151.07	/	204.71	/	152.75	/	207.00
Out-Of-Distribution (OOD)												
Py-IM	/	/	/	/	/	37.40	/	53.00	/	26.00	/	34.00
Learning3D	/	/	/	/	/	28.59	/	41.00	/	27.75	/	41.00
muzic	/	/	/	/	/	26.72	/	38.72	/	14.40	/	21.80
Grounded-SAM	/	/	/	/	177.88	48.08	271.25	67.75	177.67	62.00	271.67	88.50
Average (ID)	327.88	80.53	459.91	109.57	314.43	66.97	442.70	93.71	328.64	67.04	462.95	93.68
Average (OOD)	/	/	/	/	177.88	35.20	271.25	50.12	177.67	32.54	271.67	46.33
Total Average	327.88	80.53	459.91	109.57	291.79	60.12	414.09	84.15	303.64	59.04	431.07	84.11

B.4 Detailed Analysis of Task Variety and Instructional Depth

To provide a clearer understanding of the scope and diversity within ML-BENCH, Table 10 offers a detailed enumeration of the different types of tasks as well as an analysis of the intricacies involved in the instructions that accompany them. Each task category represents a unique section of our dataset, with Multi-Modality tasks taking the lead with 4,732 instances. Time-series and Text-related tasks follow suit with 1,478 and 1,475 instances, respectively, indicating a substantial focus on these areas as well. The numbers are counted by our 8 annotators.

Further linguistic analysis revealed the instruction sets’ complexity, with an average token length per instruction measuring 80.4 and a maximum token length reaching up to 216 tokens. Additionally, the instruction edit distance—an indicator of linguistic diversity—averages 258.7 tokens within similar tasks and 302.1 tokens across different tasks, underlining the variety and broad coverage of scenarios that ML-BENCH encompasses.

Table 10: Task distribution and instruction complexity.

Task	Number
- GNN	608
- Text	1475
- Molecular	649
- Image-GAN	1189
- Multi-Modality	4732
- Video	75
- Time-series	1478
- Attention Usage	127
- Medical	805
- 3D	264
- Music	704
Average token length per instruction	80.4
Max token length in instruction	216
Instruction edit distance among the same task	258.7
Instruction edit distance across tasks	302.1

B.5 Error Analysis for Each Repository

Figure 7 illustrates the distribution of errors made by GPT-4 across 14 repositories, categorized as per the error types described in the main text. The analysis was conducted within the context of the ML-LLM-BENCH, specifically under the Oracle setting.

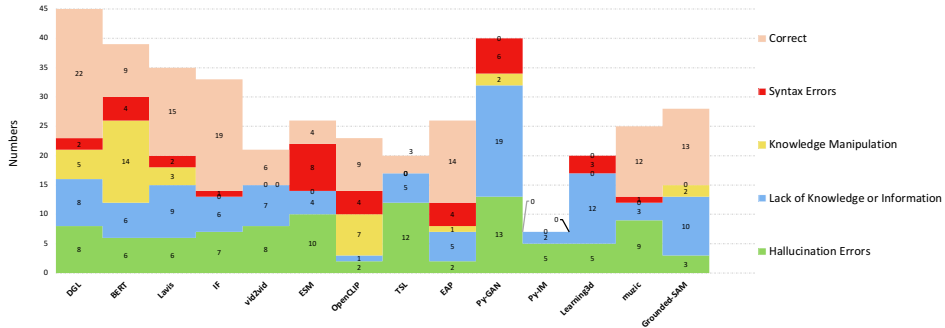


Figure 7: Using the Oracle setup, we ran GPT-4 for five iterations and tallied the number of errors across different repositories to provide an error analysis specific to each repository.

B.6 Error Analysis for ML-AGENT-BENCH

Figure 8 illustrates the distribution of errors made by OpenDevin, categorized as per the error types described in the main text. The analysis was conducted within the context of the ML-AGENT-BENCH.

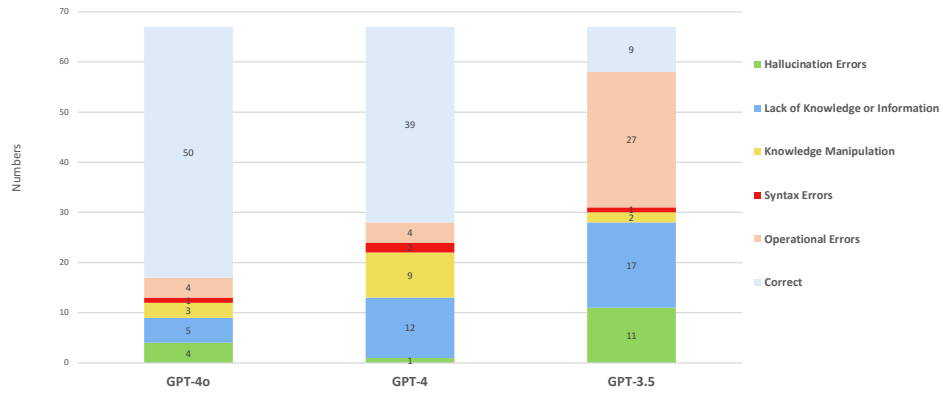


Figure 8: The error analysis for the OpenDevin framework, utilizing various base models on ML-AGENT-BENCH. Notably, Operational Error is a category unique to ML-AGENT-BENCH due to the agents' need to interact dynamically with the coding environment.

B.7 An Example of ML-LLM-BENCH

Figure 9 shows a case from ML-LLM-BENCH includes a diagram that provides a view of the model’s inputs, the reference, multiple outputs generated by the model, and an error analysis with results for each output.

Model Input	Gold Output
[readme content]: DGL is framework agnostic, meaning if a deep graph model is a component of an end-to-end application, the rest of the logics can be implemented in any major frameworks, such as PyTorch, Apache MXNet or TensorFlow.... [instruction]: I am eager to utilize the <i>Citeseer</i> dataset as the training data to empower the ARMA Model with the learning rate set to a commendably small value of <i>0.0001</i>. Additionally, I'd like to incorporate <i>5</i> stacks into this model. Your assistance in formulating the necessary code to accomplish this task would be of tremendous help. [System Prompt]: You are given [readme content], you need to carefully see [readme content] and choose wirtle code or script to implement my [instruction]. Please output code or script directly, use markdown to output code without explanation.	<pre>python citation.py --dataset Citeseer --lr 0.0001 --num-stacks 5</pre>
	Generated Result
	<pre>"python citation.py --dataset Citeseer --lr 0.01 --stacks 5", "python main.py --dataset citeseer --lr 0.0001 --stacks 5", "dgl-go --model=arma --dataset=citeseer --lr=0.0001 --stacks=5", "import dgl\ndataset = dgl.data.CiteseerGraphDataset()\ng = dataset[0]\nfrom dgllife.model import DAGNNPredictor ...", "python citation.py --dataset Citeseer --lr 0.0001 --num-stacks 5"</pre>
	Evaluation Result
	FAILED python citation.py --dataset Citeseer --lr 0.01 --stacks 5 wrong argument FAILED python citation.py --dataset citeseer --lr 0.0001 --stacks 5 no argument error FAILED dgl-go --model=arma --dataset=citeseer --lr=0.0001 --stacks=5 no file error FAILED import dgl\ndataset = dgl.data.CiteseerGraphDataset()\ng = wrong execution PASSED python citation.py --dataset Citeseer --lr 0.0001 --num-stacks 5 pass

Figure 9: The illustrative set of **input-output pairs, gold output, and execution results**, accompanied by precision metrics: **Pass@1=0** and **Pass@5=1**. Various colors within the instructions signify different parameters.

B.8 An Example of ML-AGENT-BENCH

In this section, we present detailed examples of each error type in ML-AGENT-BENCH:

1. Hallucination Errors (E1): Table 11
2. Lack of Knowledge or Information (E2): Table 12
3. Knowledge Manipulation (E3): Table 13
4. Syntax Errors (E4): Table 14
5. Operational Error (E5): Table 15

Instruction:
I have a digital image titled <code>example.jpg</code> that I hope to transform into an ink-wash-styled masterpiece with the <code>IF-I-M-v1.0</code> model. Can you guide me to write the code to transform the image accordingly?
Steps:
Navigate to the repository
<pre>cd /workspace/if/</pre>
Attempt to create the <code>run.sh</code> script in the wrong directory
The <code>run.sh</code> script is mistakenly attempted to be created in <code>/usr/bin</code> instead of the specified path: <pre>echo "#!/bin/bash" > /usr/bin/run.sh echo "python if_im.py --model_name IF-I-M-v1.0 --input_path example.jpg --output_path output.jpg" >> /usr/bin/run.sh chmod +x /usr/bin/run.sh</pre>
Error: Permission denied
The script creation fails due to insufficient permissions to write in <code>/usr/bin</code> : <pre>bash: /usr/bin/run.sh: Permission denied</pre>
Error: Script not created and task not executed
The task is not executed because the script could not be created in the specified path: <pre>The run.sh script could not be created in /usr/bin due to permission issues. The task in /workspace/if/ was not executed.</pre>

Table 11: An example of E1 error. Task execution attempt for Zero-shot Image-to-Image Translation. The task could not be completed due to an attempt to create the script in `/usr/bin`, which resulted in permission denial, and the intended task was not executed.

Instruction:
Can you assist me in writing the code to extract 5 designs from the structure Fam20A.pdb, specifically from chain B, at a temperature of 8?
Steps:
Navigate to the repository
<code>cd /workspace/esm/</code>
Create the run.sh script
The run.sh script is created with the following content:
<pre>#!/bin/bash python main.py --input Fam20A.pdb --chain B --temperature 8 --output designs.txt --num_designs 5</pre>
Run the script
<code>./run.sh</code>
Error: Missing main.py script
The execution of run.sh fails due to the usage of main.py instead of extract_designs.py:
<code>python: cannot open file main.py: [Errno 2] No such file or directory</code>
Check for the missing script
The user is asked to ensure that the main.py script is present in the repository:
Please make sure that the main.py script is available in the /workspace/esm/ repository.
Task cannot be completed
The task cannot be completed without the main.py script:
I cannot complete the task as requested without the missing script.

Table 12: An example of E2 error. Task execution attempt for extracting designs from Fam20A.pdb. The task could not be completed due to the misinformation of main.py script. The LLM agent should use extract_designs.py instead.

Instruction:
Looking to use the Cluster GAN model for a project. Hoping to set up a learning rate of 0.0001 and plan on running it for 100 epochs. I would like the batch size to be 16 and to keep the image size at 128. Can you help me craft some code?
Steps:
Create the run.sh script
The run.sh script is created with the following content:
<pre>#!/bin/bash python main.py --model cluster_gan --learning_rate 0.0001 --num_epochs 100 --batch_size 16 --image_size 128</pre>
Run the script
<pre>cd /workspace/PyTorch-GAN/implementations/cluster_gan/ chmod +x run.sh ./run.sh</pre>
Error: Missing main.py script
The execution of run.sh fails due to the missing main.py script:
<pre>python: cannot open file /workspace/PyTorch-GAN/implementations/ cluster_gan/main.py: [Errno 2] No such file or directory</pre>
Update run.sh script to use clustergan.py
The run.sh script is updated with the following content:
<pre>#!/bin/bash python clustergan.py --model cluster_gan --learning_rate 0.0001 -- num_epochs 100 --batch_size 16 --image_size 128</pre>
Run the updated script
<pre>cd /workspace/PyTorch-GAN/implementations/cluster_gan/ chmod +x run.sh ./run.sh</pre>
Error: Unrecognized arguments in clustergan.py
The execution of run.sh fails due to unrecognized arguments in clustergan.py:
<pre>usage: clustergan.py [-h] [-n N_EPOCHS] [-b BATCH_SIZE] [-i IMG_SIZE] [-d LATENT_DIM] [-l LEARNING_RATE] [-c N_CRITIC] [- w] clustergan.py: error: unrecognized arguments: --model cluster_gan -- learning_rate 0.0001 --num_epochs 100 --image_size 128</pre>

Table 13: An example of E3 error. Task execution attempt for Cluster GAN model. The task could not be completed due to unrecognized arguments in clustergan.py. The LLM agent should have ensured the script accepts the required arguments.

Instruction:
Please complete the Machine Learning task in the following repository: esm The task is: Sample sequences design for certain structure Can you assist me in writing the code to extract 3 structure examples from the inverse_folding data file 5YH2.pdb? I specifically need designs with chain A at a temperature of 5. The output should be saved in the model_structure.fasta file. You should create a script named run.sh under the specified path in the repo to run the task. You can find the task repo at: /workspace/esm/ You should terminate the subprocess after running the task (e.g., call subprocess.Popen(args).wait()). When you think you have completed the task, please run the following command: <execute_bash> exit </execute_bash>.
Steps:
Create the run.sh file using Python code
Attempt to create the run.sh file using Python code fails due to a syntax error: Running Python code interactively: <code>create_file('/workspace/esm/run.sh')</code> <pre> File "/opendevin/plugins/jupyter/execute_cli.py", line 34 if jupyter_pwd := os.environ.get('JUPYTER_PWD'): ^ SyntaxError: invalid syntax </pre>
Task cannot be completed
The task cannot be completed due to the syntax error in the Python code: I cannot complete the task as requested due to a syntax error in the Python code used to create the run.sh file.

Table 14: An example of E4 error. Task execution attempt for Sample sequences design for certain structure. The task could not be completed due to a syntax error in the Python code used to create the run.sh file.

Instruction:
Can you assist me in writing the code to extract 3 designs from the structure 5YH2.pdb, specifically from chain B, at a temperature of 1? The output should be saved to the file model_structure.fasta.
Steps:
Navigate to the repository
<code>cd /workspace/esm/</code>
Ask the user to create a run.sh
Now let's create the run.sh script to extract the designs from the structure. Please create a new file named run.sh in the esm repository and add the following code to it:
<pre>#!/bin/bash python extract_designs.py --structure 5YH2.pdb --chain B --temperature 1 --output model_structure.fasta</pre>
Run command
<code>./run.sh</code>
<pre>bash: /workspace/esm/run.sh: No such file or directory ERROR conda.cli.main_run:execute(47): (*@'conda run bash /workspace/esm/ run.sh'@*) failed. (See above for error)</pre>
Ask the user to create a run.sh
...
Failed after so many iterations
<pre>bash: /workspace/esm/run.sh: No such file or directory ERROR conda.cli.main_run:execute(47): (*@'conda run bash /workspace/esm/ run.sh'@*) failed. (See above for error)</pre>

Table 15: An example of E5 error. Task execution attempt for extracting designs from 5YH3.pdb. The LLM agent mistakenly asks the human for help and refuses to do the task by itself.

C Experiment Details

C.1 Efficacy of BM25 Retrieval on Repository-Level Code

When evaluating the relevance of retrieved code segments within repository-level tasks, we utilized the BLEU-1 score as a metric to measure the similarity between hand-annotated "Oracle" segments and those extracted by the model using BM25 retrieval. "Oracle" segments are those identified by annotators as most pertinent, serving as the foundational evidence required for the task at hand. As depicted in Table 16, the resulting BLEU scores indicate a low degree of correlation, suggesting that the retrieval segments identified by BM25 are significantly dissimilar to the Oracles crafted by human annotators. This finding is demonstrative of BM25’s limitations in effectively identifying the most relevant content for repository-scale code, as evidenced by the low BLEU scores.

Table 16: The BLEU-1 scores between Oracles and the Retrieval content.

	ID-train	OOD-train	ML-BENCH
BLEU score	0.0112	0.0087	0.0082

C.2 Information Lossing due to Truncation

It is reasonable that truncation may lead to information missing, but it is worth noting that only in the Code setting for the open-source models does the input of README files need to be truncated to 8k, which is inevitable because of the input length limitation. However, only a small number of README files need to be truncated. To qualitatively present the information loss percentage due to truncation, we present the percentage of losing critical information during truncation in Table 17. Note that all the results are manually examined. We can identify that only 5 repositories lose critical information after truncating the README files.

Table 17: The percentage of losing critical information due to truncation.

Repos	Proportion of losing information (%)
vid2vid	0
If	0
DGL	0
Py-GAN	33.3
ESM	11.76
BERT	100
OpenCLIP	0
Lavis	0
TSL	0
EAP	75
Grounded-SAM	0
Py-IM	20
muzic	0
Learning3d	0
SD	0
MedZooPy	0
TCL	0
Tensor2Tensor	0
Total	0

C.3 Examples of Input-output of each GitHub Repository

In this section, we present detailed examples of the input and output of each GitHub Repo in Tab.18 to Tab.35. The corresponding repository for each table is shown below:

1. External-Attention: Table 18
2. BERT: Table 19
3. Deep learning on graphs: Table 20
4. Evolutionary scale modeling: Table 21
5. Grounded-Segment-Anything: Table 22
6. DeepFloyd IF: Table 23
7. Language-Vision Intelligence: Table 24
8. Deep learning on 3D point clouds data: Table 25
9. 3D multi-modal medical image segmentation library: Table 26
10. Music understanding and generation: Table 27
11. Implementation of OpenAI's CLIP: Table 28
12. Generative Adversarial Network varieties: Table 29
13. PyTorch Image Models: Table 30
14. Stable diffusion: Table 31
15. Text classification: Table 32
16. Tensor2Tensor: Table 33
17. deep time series analysis: Table 34
18. Video-to-video translation: Table 35

README:
As a supplement to the project, an object detection codebase, YOLO. Air has recently been opened. It integrates various attention mechanisms in the object detection algorithm. The code is simple and easy to read. Welcome to play and star! For beginners (like me): Recently, I found a problem when reading the paper. Sometimes the core idea of the paper is very simple, and the core code may be just a dozen lines. However, when I open the source code of the author's release, I find that the proposed module is embedded in the task framework such as classification, detection, and segmentation, resulting in redundant code. For me who is not familiar with the specific task framework, it is difficult to find the core code, resulting in some difficulties in understanding the paper and network ideas. For advanced (like you): If the basic units conv, FC, and RNN are regarded as small Lego blocks, and the structures transformer and RESNET are regarded as LEGO castles that have been built, the modules provided by this project are LEGO components with complete semantic information. To avoid repeatedly building wheels, scientific researchers should just think about how to use these "LEGO components" to build more colorful works. For proficient (maybe like you): Limited capacity, do not like light spraying!!! For All: This project aims to realize a code base that can make beginners of deep learning understand and serve scientific research and industrial communities. As fightingcv WeChat official account. The purpose of this project is to achieve Let there be no hard-to-read papers in the world. (at the same time, we also welcome all scientific researchers to sort out the core code of their work into this project, promote the development of the scientific research community, and indicate the author of the code in README) ...
Oracle Segment:
<pre>.. from model.attention.ViP import WeightedPermuteMLP import torch from torch import nn from torch.nn import functional as F ...</pre>
Instruction:
I'm planning to utilize the fighting-cv model to complete the attention layers for ViP Attention Usage. Could you provide me with some guidance on accomplishing this task?
Instruction:
<pre>package_1: ViP sub_package: WeightedPermuteMLP package_2: torch</pre>
Ground Truth Output:
<pre>from model.attention.ViP import WeightedPermuteMLP import torch from torch import nn from torch.nn import functional as F input=torch.randn(64,8,8,512) seg_dim=8 vip=WeightedPermuteMLP(512,seg_dim) out=vip(input) print(out.shape)</pre>

Table 18: Example of input-output for **External-Attention-pytorch** GitHub on **attention layer** task on **Attention Usage** domain. The README URL is https://github.com/xmu-xiaoma666/External-Attention-pytorch/blob/master/README_EN.md. The GitHub URL is <https://github.com/xmu-xiaoma666/External-Attention-pytorch>.

README:
BERT
New March 11th, 2020: Smaller BERT Models This is a release of 24 smaller BERT models (English only, uncased, trained with WordPiece masking) referenced in Well-Read Students Learn Better: On the Importance of Pre-training Compact Models. ...
Oracle Segment:
..
This demo code only pre-trains for a small number of steps (20), but in practice you will probably want to set 'num_train_steps' to 10000 steps or more. The 'max_seq_length' and 'max_predictions_per_seq' parameters passed to 'run_pretraining.py' must be the same as 'create_pretraining_data.py'.
python run_pretraining.py --input_file=/tmp/tf_examples.tfrecord --output_dir=/tmp/pretraining_output --do_train=True --do_eval=True --bert_config_file=\$BERT_BASE_DIR/bert_config.json --init_checkpoint=\$BERT_BASE_DIR/bert_model.ckpt ...
Instruction:
Behold, a formidable quest awaits - the pre-training of the unparalleled uncased_L-24_H-1024_A-16 model. Our path to victory lies in configuring the maximum sequence length to a mighty 512, with a pledge to uphold the limit of 30 predictions per sequence. Battling through the treacherous terrain of 10000 steps, we shall march forward, with a stalwart battalion of 32 batch size by our side. But fear not, for we shall brace ourselves with 10000 warmup steps, as we navigate the perilous sea of learning, with a steadfast learning rate of 0.0001. I humbly beseech your assistance, noble comrade, to conjure the code necessary to conquer this heroic endeavor.
Arguments Requirements:
model: ./model/uncased_L-12_H-768_A-16 train_batch_size: 32 max_seq_length: 512 num_train_steps: 10000 num_warmup_steps: 1000 learning_rate: 0.0001
Ground Truth Output:
python run_pretraining.py --input_file=/tmp/tf_examples.tfrecord --output_dir=/tmp/pretraining_output --do_train=True --do_eval=True --bert_config_file=./model/uncased_L-24_H-1024_A-16/bert_config.json --init_checkpoint=./model/uncased_L-24_H-1024_A-16/bert_model.ckpt --train_batch_size=32 --max_seq_length=512 ...

Table 19: Example of input-output for **bert** GitHub on **pre-training** task on **BERT** domain. The README URL is <https://github.com/google-research/bert/blob/master/README.md>. The GitHub URL is <https://github.com/google-research/bert>.

README:
1. DGL Implementation of CorrectAndSmooth This DGL example implements the GNN model proposed in the paper Combining Label Propagation and Simple Models Out-performs Graph Neural Networks. For the original implementation, see here. Contributor: xnuohz
2. Requirements The codebase is implemented in Python 3.7. For version requirement of packages, see below. dgl 0.6.0.post1 torch 1.7.0 ogb 1.3.0 ...
Oracle Segment:
.. 3.1 ogbn-arxiv Plain MLP + C&S python main.py --dropout 0.5 python main.py --pretrain --correction-adj DA --smoothing-adj AD --autoscale ...
Instruction:
... Together, we shall embark on a noble mission to train the illustrious CorrectAndSmooth Model, fortified with a sublime dropout rate of 0.7. Our arduous journey spans 700 epochs, each pulsating with the promise of enlightenment. Alas, I beseech your sage guidance in the ethereal realm of code crafting, to manifest this grand undertaking.
Arguments Requirements:
dataset: ogbn-arxiv model: mlp dropout: 0.7 epochs: 700
Ground Truth Output:
python main.py --dataset ogbn-arxiv --model mlp --dropout 0.7 --epochs 700

Table 20: Example of input-output for **DGL** GitHub on **DGL Implementation of CorrectAndSmooth** task on **GNN** domain. The README URL is https://github.com/dmlc/dgl/blob/master/examples/pytorch/correct_and_smooth/README.md. The GitHub URL is <https://github.com/dmlc/dgl>.

README:
Evolutionary Scale Modeling atlas Update April 2023: Code for the two simultaneous preprints on protein design is now released! Code for "Language models generalize beyond natural proteins" is under examples/lm-design/. Code for "A high-level programming language for generative protein design" is under examples/protein-programming-language This repository contains code and pre-trained weights for Transformer protein language models from the Meta Fundamental AI Research Protein Team (FAIR), including our state-of-the-art ESM and ESMFold, as well as MSA Transformer, ESM-1v for predicting variant effects and ESM-IF1 for inverse folding. ...
Oracle Segment:
... The following commands allow the extraction of the final-layer embedding for a FASTA file from the ESM-2 model: <pre>esm-extract esm2_t33_650M_UR50D examples/data/some_proteins.fasta examples/data/some_proteins_emb_esm2 --repr_layers 0 32 33 --include</pre> <pre>python scripts/extract.py esm2_t33_650M_UR50D examples/data/some_proteins.fasta examples/data/some_proteins_emb_esm2 --repr_layers 0 32 33 --include mean per_tok --A cuda device is optional and will be auto-detected.</pre> ...
Instruction:
... Can you assist me in writing the code to extract the 24-layer embedding for a FASTA file named rna.fasta using the esm1v_t33_650M_UR90S_5 model and save the output?
Arguments Requirements:
model: esm1v_t33_650M_UR90S_5 data: rna.fasta layer_number: 24 layer_name: repr_layers
Ground Truth Output:
<pre>python scripts/extract.py esm1v_t33_650M_UR90S_5 rna.fasta output.embeddings --repr_layers 24 --include mean per_tok</pre>

Table 21: Example of input-output for **ESM** GitHub on **Extract ESMFold Structure Prediction Model’s Embedding** task on **molecular** domain. The README URL is <https://github.com/facebookresearch/esm/blob/master/README.md>. The GitHub URL is <https://github.com/facebookresearch/esm>.

README:
Official PyTorch implementation of Grounding DINO), a stronger open-set object detector. Code is available now!
Highlight
- Open-Set Detection. Detect everything with language! - High Performance. COCO zero-shot 52.5 AP (training without COCO data!). COCO fine-tune 63.0 AP. - Flexible. Collaboration with Stable Diffusion for Image Editing.
...
Oracle Segment:
<pre> .. Demo python demo/inference_on_a_image.py -c /path/to/config -p /path/to/checkpoint -i .asset/cats.png -o outputs/0 -t cat ear. [--cpu-only] # open it for cpu mode See the demo/inference_on_a_image.py for more details. </pre>
...
Instruction:
<pre> .. I am interested in utilizing the grounding dino demo for a specific task. The input image path is ground_segment/GD_new.json, and I would like the output to be saved in the directory output/- cat2002. Additionally, I would like the text condition to be set to right ear of cat. Could you kindly assist me in writing the script to achieve this? </pre>
Arguments Requirements:
<pre> i: .asset/cat.jpg o: output/cat2002 t: right ear of cat </pre>
Ground Truth Output:
<pre> python demo/inference_on_a_image.py -c model/GroundingDINO_SwinT_OGC.py -p model/groundingdino_swint_ogc.pth -i .asset/cat.jpg -o output/cat2002 -t right ear of cat </pre>

Table 22: Example of input-output for **Grounded-Segment-Anything** GitHub on **Grounding DINO** demo task on **Segment** domain. The README URL is <https://github.com/IDEA-Research/Grounded-Segment-Anything/blob/main/GroundingDINO/README.md>. The GitHub URL is <https://github.com/IDEA-Research/Grounded-Segment-Anything>.

README:
We introduce DeepFloyd IF, a novel state-of-the-art open-source text-to-image model with a high degree of photorealism and language understanding. DeepFloyd IF is a modular composed of a frozen text encoder and three cascaded pixel diffusion modules: a base model that generates 64x64 px image based on text prompt and two super-resolution models, each designed to generate images of increasing resolution: 256x256 px and 1024x1024 px. All stages of the model utilize a frozen text encoder based on the T5 transformer to extract text embeddings, which are then fed into a UNet architecture enhanced with cross-attention and attention pooling. The result is a highly efficient model that outperforms current state-of-the-art models, achieving a zero-shot FID score of 6.66 on the COCO dataset. Our work underscores the potential of larger UNet architectures in the first stage of cascaded diffusion models and depicts a promising future for text-to-image synthesis. ...
Oracle Segment:
.. II. Zero-shot Image-to-Image Translation In Style Transfer mode, the output of your prompt comes out at the style of the support_pil_img from deepfloyd_if.pipelines import style_transfer result = style_transfer(t5 = t5, if_I = if_I, if_II = if_II, ...
Instruction:
... Time to create a visual masterpiece! I am excited to recreate 'image'.jpg into a fascinating rendition of the ink wash style. I am planning to utilize the capabilities of the IF-I-XL-v1.0 model for this endeavor. Can you assist me in crafting the necessary code?
Arguments Requirements:
model: IF-I-XL-v1.0 argument1: image.jpg argument2: wash
Ground Truth Output:
<pre>from deepfloyd_if.modules import IFStageI, IFStageII, StableStageIII from deepfloyd_if.modules.t5 import T5Embedder device = 'cuda:1' if_I = IFStageI('IF-I-XL-v1.0', device=device) if_II = IFStageII('IF-II-L-v1.0', device=device) if_III = StableStageIII('stable-diffusion-x4-upscaler', device=device) t5 = T5Embedder(device='cpu')</pre> style_prompt = a captivating ink wash style image_path = image.jpg ...

Table 23: Example of input-output for **IF** GitHub on **Zero-shot Image-to-Image Translation** task on **Image and Text** domain. The README URL is <https://github.com/deep-floyd/IF/blob/develop/README.md>. The GitHub URL is <https://github.com/deep-floyd/if>.

README:
Lavis Lavis - A Library for Language-Vision Intelligence What's New: A simple, yet effective, cross-modality framework built atop frozen LLMs that allows the integration of various modalities (image, video, audio, 3D) without extensive modality-specific customization. Technical Report and Citing LAVIS: ...
Oracle Segment:
... how to use models in LAVIS to perform inference on example data. We first load a sample image from local. import torch from PIL import Image # setup device to use device = torch.device("cuda" if torch.cuda.is_available() else "cpu") # load sample image raw_image = Image.open(merlion.png).convert(RGB) This example image shows Merlion park (source), a landmark in Singapore. # Image Captioning In this example, we use the BLIP model to generate a caption for the image. ...
Instruction:
There is a task in front of me currently, which is the Image Captioning task. I ask for your kind help in moving forward with this task.
Arguments Requirements:
image: image.jpg
Ground Truth Output:
import torch from PIL import Image from lavis.models import load_model_and_preprocess device = torch.device('cuda:1' if torch.cuda.is_available() else 'cpu') model, vis_processors, _ = load_model_and_preprocess(name='blip_caption', model_type='base_coco', is_eval=True, device=device) raw_image = Image.open('image.jpg').convert('RGB') preprocessed_image = vis_processors['eval'](raw_image).unsqueeze(0).to(device) output = model.generate({'image': preprocessed_image}) caption = output['captions'][0]['text'] print(caption)

Table 24: Example of input-output for **Lavis** GitHub on **Image Captioning** task on **Multimodal Image and Text** domain. The README URL is <https://github.com/salesforce/LAVIS/blob/main/README.md>. The GitHub URL is <https://github.com/salesforce/lavis>.

README:
Learning3D: A Modern Library for Deep Learning on 3D Point Clouds Data. Learning3D is an open-source library that supports the development of deep learning algorithms that deal with 3D data. The Learning3D exposes a set of state of art deep neural networks in python. A modular code has been provided for further development. We welcome contributions from the open-source community. Available Computer Vision Algorithms in Learning3D ...
Oracle Segment:
... examples/test_dcp.py Learning3D is an open-source library that supports the development of deep learning algorithms that deal with 3D data. The Learning3D exposes a set of state of art deep neural networks in python python test_dcp.py --num_points 128 --j 12 --symfn max ...
Instruction:
I am interested in conducting a test using the dcp model. Specifically, I would like to set the parameters as follows: the test mode should be selected, the model should be set to dcp, the number of points should be 512, the number of data loading workers should be -j 8, and the symmetric function should be set to -symfn max. Could you please assist me in writing the code or script necessary to carry out this test?
Arguments Requirements:
number of points: 512 number of data loading workers: 8 symmetric function: max
Ground Truth Output:
python test_dcp.py --num_points 512 --j 8 --symfn max

Table 25: Example of input-output for **Learning3D** GitHub on **Test dcp model** task on **3D** domain. The README URL is <https://github.com/vinits5/learning3d/blob/master/README.md>. The GitHub URL is <https://github.com/vinits5/learning3d>.

README:
MusicBERT Basics All models accept two parameters: a) the input the channels (in_channels), and b) the segmentation classes (classes) and produce un-normalized outputs All losses accept as input the prediction in 5D shape of [batch,classes,dim_1,dim_2,dim_3] and the target in 4D target shape of [batch, dim_1, dim_2, dim_3]. It is converted to one-hot inside the loss function for consistency reasons. Furthermore the normalization of the predictions is handled here. Dice-based losses return the scalar loss for backward(), and the prediction per channels in numpy to track training progress. ...
Oracle Segment:
Usage How to train your model For Iseg-2017 : python ./examples/train_iseg2017_new.py --args For MR brains 2018 (4 classes) python ./examples/train_mrbrains_4_classes.py --args For MR brains 2018 (8 classes) python ./examples/train_mrbrains_9_classes.py --args For MICCAI 2019 Gleason Challenge python ./examples/test_miccai_2019.py --args The arguments that you can modify are extensively listed in the manual.
Instruction:
I'm seeking assistance in writing a piece of code that can successfully train a model for the 'Iseg 2017 Task'. The model in question is 'RESNET3DVAE' and I require the learning rate to be set to '1e-3'. It is also crucial that the training samples are set to '10'. Lastly, use 'sgd' as the optimizer. Could you kindly help out in creating this algorithm?
Arguments Requirements:
lr: 1e-3 samples_train: 10 model: RESNET3DVAE optimizer: sg
Ground Truth Output:
python ./examples/train_iseg2017_new.py --lr 1e-3 --samples_train 10 --model RESNET3DVAE --opt sgd

Table 26: Example of input-output for **MedicalZoo** GitHub on **Iseg-2017** task on **Medical** domain. The README URL is <https://github.com/black0017/MedicalZooPytorch/blob/master/manual/README.md>. The GitHub URL is <https://github.com/black0017/MedicalZooPytorch>.

README:
MusicBERT MusicBERT: Symbolic Music Understanding with Large-Scale Pre-Training, by Mingliang Zeng, Xu Tan, Rui Wang, Zeqian Ju, Tao Qin, Tie-Yan Liu, ACL 2021, is a large-scale pre-trained model for symbolic music understanding. It has several mechanisms including OctupleMIDI encoding and bar-level masking strategy that are specifically designed for symbolic music data, and achieves state-of-the-art accuracy on several music understanding tasks, including melody completion, accompaniment suggestion, genre classification, and style classification. Projects using MusicBERT: midiformers: a customized MIDI music remixing tool with easy interface for users.
1. Preparing datasets
1.1 Pre-training datasets
Prepare tar -xzvf lmd_full.tar.gz zip -r lmd_full.zip lmd_full Run the dataset processing script. ('preprocess.py') python -u preprocess.py The script should prompt you to input the path of the midi zip and the path for OctupleMIDI output. ...
Oracle Segment:
Pre-training bash train_mask.sh lmd_full small Download our pre-trained checkpoints here: small and base, and save in the checkpoints folder. (a newer version of fairseq is needed for using provided checkpoints: see issue-37 or issue-45)
Instruction:
I am interested in conducting a test using the dcp model. Specifically, I would like to set the parameters as follows: the test mode should be selected, the model should be set to dcp, the number of points should be 512, the number of data loading workers should be -j 8, and the symmetric function should be set to -symfn max. Could you please assist me in writing the code or script necessary to carry out this test?
Arguments Requirements:
bash: train_mask.sh dataset: lmd_full checkpoint: small
Ground Truth Output:
bash train_mask.sh lmd_full small

Table 27: Example of input-output for **Muzic** GitHub on **Pre-training model** task on **Music** domain. The README URL is <https://github.com/microsoft/muzic/blob/main/musicbert/README.md>. The GitHub URL is <https://github.com/microsoft/muzic>.

README:
OpenCLIP Welcome to an open-source implementation of OpenAI's CLIP(Contrastive Language-Image Pre-training). Using this codebase, we have trained several models on a variety of data sources and compute budgets, ranging from small-scale experiments to larger runs including models trained on datasets such as LAION-400M, LAION-2B and DataComp-1B. Many of our models and their scaling properties are studied in detail in the paper reproducible scaling laws for contrastive language-image learning. Some of our best models and their zero-shot ImageNet-1k accuracy are shown below, along with the ViT-L model trained by OpenAI. We provide more details about our full collection of pretrained models here, and zero-shot results for 38 datasets here. midiformers: a customized MIDI music remixing tool with easy interface for users. ...
Oracle Segment:
Pre-training bash train_mask.sh lmd_full small Download our pre-trained checkpoints here: small and base, and save in the checkpoints folder. (a newer version of fairseq is needed for using provided checkpoints: see issue-37 or issue-45)
Instruction:
I'm hoping you could assist me with a task. I need to sort a collection of images located at the directory './image'. For this purpose, I'll be using the 'ViT-B-32' model along with the pretraining model 'commonpool_s_laion_s13m_b4k'. Would you be able to provide instructions or steps on how to accomplish this task?
Arguments Requirements:
model: ViT-B-32 pretraining_model: commonpool_s_laion_s13m_b4k path: ./image
Ground Truth Output:
import os import numpy as np import torch import open_clip open_clip.list_pretrained() model, _, preprocess = open_clip.create_model_and_transforms('ViT-B-32', pretrained='commonpool_s_laion_s13m_b4k') model.eval() context_length = model.context_length vocab_size = model.vocab_size ...

Table 28: Example of input-output for **OpenCLIP** GitHub on **Image classification** task on **Image and Text** domain. The README URL is https://github.com/mlfoundations/open_clip/blob/main/README.md. The GitHub URL is https://github.com/mlfoundations/open_clip.

README:
MusicBERT Collection of PyTorch implementations of Generative Adversarial Network varieties presented in research papers. Model architectures will not always mirror the ones proposed in the papers, but I have chosen to focus on getting the core ideas covered instead of getting every layer configuration right. Contributions and suggestions of GANs to implement are very welcomed. Table of Contents: Installation Implementations Auxiliary Classifier GAN Adversarial Autoencoder ...
Oracle Segment:
... Energy-Based GAN Among them, we show one instantiation of EBGAN framework as using an auto-encoder architecture, with the energy being the reconstruction error, in place of the discriminator. We show that this form of EBGAN exhibits more stable behavior than regular GANs during training. We also show that a single-scale architecture can be trained to generate high-resolution images. Run Example \$ cd implementations/ebgan/ \$ python3 ebgan.py
Instruction:
I have a task to work with the Energy-Based GAN model. The learning rate for this task needs to be set at 0.0001, the number of training epochs should be defined as 100, and the batch size should be fixed at 16. Furthermore, I want the image size to be set at 128. Can you please assist me in framing the script to facilitate this?
Arguments Requirements:
lr: 0.0001 n_epochs: 100 batch_size: 16 img_size: 128 model: ebgan
Ground Truth Output:
python3 ebgan.py --lr 0.0001 --n_epochs 100 --batch_size 16 --mg_size 128

Table 29: Example of input-output for **pyGAN** GitHub on **Energy-Based GAN** task on **images-many-GANs** domain. The README URL is <https://github.com/eriklindernoren/PyTorch-GAN/blob/master/README.md>. The GitHub URL is <https://github.com/eriklindernoren/PyTorch-GAN>.

README:
PyTorch Image Models
...
What's new
...
Introduction
PyTorch Image Models (timm) is a collection of image models, layers, utilities, optimizers, schedulers, data-loaders / augmentations, and reference training / validation scripts that aim to pull together a wide variety of SOTA models with ability to reproduce ImageNet training results.
...
Oracle Segment:
..
Existing method of changing patch_size (resize pretrained patch_embed weights once) on creation still works.
Example validation cmd
python validate.py /imagenet
--model vit_base_patch16_224
--amp
--amp-dtype bfloat16
--img-size 255
--crop-pct 1.0
--model-kwarg dynamic_img_size=True dynamic_img_pad=True
...
Instruction:
I am interested in performing the task of resizing the image or window. For this purpose, I would like to utilize the model vit_base_patch16_224. Additionally, it would be helpful to set the amp-dtype to bfloat16. Moreover, I would like to specify the image size as 255 and the crop percentage as 1.0. To ensure flexibility, I would like to enable dynamic image size and dynamic image padding. Could you kindly assist me in creating the code or script to accomplish this objective?
Arguments Requirements:
model: vit_base_patch16_224
amp-dtype: bfloat16
img-size: 255
crop-pct: 1.0
dynamic_img_size: True
dynamic_img_pad: True
Ground Truth Output:
python validate.py /imagenet
--model vit_base_patch16_224
--amp
--amp-dtype bfloat16
--img-size 255
--crop-pct 1.0
--model-kwarg dynamic_img_size=True

Table 30: Example of input-output for **PyIM** GitHub on **PyIM Implementation of Resize The Image/Window** task on **Image** domain. The README URL is <https://github.com/huggingface/pytorch-image-models/blob/main/README.md>. The GitHub URL is <https://github.com/huggingface/pytorch-image-models>.

README:
Stable Diffusion Version 2 This repository contains Stable Diffusion models trained from scratch and will be continuously updated with new checkpoints. The following list provides an overview of all currently available models. More coming soon. ... Requirements You can update an existing latent diffusion environment by running. ...
Oracle Segment:
... We provide the configs for the SD2-v (768px) and SD2-base (512px) model. First, download the weights for SD2.1-v and SD2.1-base. To sample from the SD2.1-v model, run the following: <pre>python scripts/txt2img.py --prompt "a professional photograph of an astronaut riding a horse" --ckpt <path/to/768model.ckpt> --config configs/stable-diffusion/v2-inference-v.yaml --H 768 --W 768</pre> or try out the Web Demo: Hugging Face Spaces. ...
Instruction:
... For the task of generating an image from text, I need your assistance in writing the code. We'll be using the scripts/txt2img.py script along with the SD2.1-v model. Ensure that the model checkpoint file is located at As we want to generate a high-quality image, set the number of sampling steps to 20. The prompt to generate the image is "a professional photograph of an astronaut riding a horse" and we only need one iteration of the generation process. Can you help me write the code to accomplish this task?
Arguments Requirements:
<pre>repeat: 1 config: "configs/stable-diffusion/v2-inference-v.yaml" ckpt: "ckpt/SD2_1_v_model.ckpt" prompt: "a professional photograph of an astronaut riding a horse" precision: full steps: 20 seed: 2048</pre>
Ground Truth Output:
<pre>python scripts/txt2img.py --prompt "a professional photograph of an astronaut riding a horse" --ckpt ckpt/SD2_1_v_model.ckpt --config configs/stable-diffusion/v2-inference-v.yaml --H 768 --W 768 --seed 2048 --precision full --steps 20 --repeat 1</pre>

Table 31: Example of input-output for **SD** GitHub on **SD Implementation of Text-to-Image** task on **Stable Diffusion** domain. The README URL is <https://github.com/Stability-AI/stablediffusion/blob/main/README.md>. The GitHub URL is <https://github.com/Stability-AI/stablediffusion>.

README:
Text Classification
The purpose of this repository is to explore text classification methods in NLP with deep learning.
...
Usage:
1.model is in xxx_model.py
2.run python xxx_train.py to train the model
...
Oracle Segment:
it learn representation of each word in the sentence or document with left side context and right side context:
representation current word=[left_side_context_vector,current_word_embedding,right_side_context_vecotor].
for left side context, it use a recurrent structure, a no-linearity transfrom of previous word and left side previous context; similarly to right side context.check: p71_TextRCNN_model.py
Instruction:
I am looking to utilize the TextRCNN model for a particular task. In the course of executing this task, I would like to fix the learning rate at 0.00001, the number of training epochs at 300, and set my batch size to 16. Are you in a position to assist me in creating the appropriate coding syntax for this purpose?
Arguments Requirements:
model: TextRCNN
learning_rate: 0.00001
num_epochs: 300
batch_size: 16
Ground Truth Output:
python3 a04_TextRCNN/p71_TextRCNN_train.py
--num_epochs 300
--batch_size 16
--lr 0.00001

Table 32: Example of input-output for **TC** GitHub on **TC Implementation of TextRCNN** task on **texts** domain. The README URL is https://github.com/brightmart/text_classification/blob/master/README.md. The GitHub URL is https://github.com/brightmart/text_classification.

README:
Tensor2Tensor Tensor2Tensor, or T2T for short, is a library of deep learning models and datasets designed to make deep learning more accessible and accelerate ML research. ... Quick Start This iPython notebook explains T2T and runs in your browser using a free VM from Google, no installation needed. Alternatively, here is a one-command version that installs T2T, downloads MNIST, trains a model and evaluates it: ...
Oracle Segment:
... Test the quick-start on a Workspace's Terminal with this command <pre>t2t-trainer --generate_data --data_dir=./t2t_data --output_dir=./t2t_train/mnist --problem=image_mnist --model=shake_shake --hparams_set=shake_shake_quick --train_steps=1000 --eval_steps=100</pre> Note: Ensure compliance with the FloydHub Terms of Service. ...
Instruction:
I'd like assistance in trying to make sense of mathematical language. Any help you can give would be highly appreciated, in comprehending the intricate nature of mathematical terminologies and symbols.
Ground Truth Output:
<pre>t2t-trainer --generate_data --data_dir= /t2t_data --output_dir= /t2t_train/mathematical_language --problem=algorithmic_math_two_variables --model=transformer --hparams_set=transformer_tiny --train_steps=1000 --eval_steps=100</pre>

Table 33: Example of input-output for **tensor2** GitHub on **tensor2 Implementation of Mathematical Language Understanding** task on **Text** domain. The README URL is <https://github.com/tensorflow/tensor2tensor/blob/master/README.md>. The GitHub URL is <https://github.com/tensorflow/tensor2tensor>.

README:
Time Series Library (TSlib) TSlib is an open-source library for deep learning researchers, especially for deep time series analysis. We provide a neat code base to evaluate advanced deep time series models or develop your model, which covers five mainstream tasks: long- and short-term forecasting, imputation, anomaly detection, and classification. ... Usage 1. Install Python 3.8. For convenience, execute the following command. pip install -r requirements.txt ... Oracle Segment: ... Train and evaluate model. We provide the experiment scripts for all benchmarks under the folder ./scripts/. You can reproduce the experiment results as the following examples: long-term forecast bash ./scripts/long_term_forecast/ETT_script/TimesNet_ETTh1.sh short-term forecast bash ./scripts/short_term_forecast/TimesNet_M4.sh ... Instruction: I'm aiming to conduct a performance evaluation of my FEDformer model which is designated for a classification task. I would like to utilize subset: FaceDetection of the UEA benchmark dataset for this evaluation. The patience parameter needs to be set to 10, with the train epochs set to 16, and batch size adjusted to 100. Could you kindly provide me the code necessary to accomplish this? Arguments Requirements: task_name: classification model: FEDformer data: UEA model_id: FaceDetection patience: 10 batch_size: 16 train_epochs: 100 Ground Truth Output: python -u run.py --task_name classification --model FEDformer --data UEA --e_layers 3 --batch_size 16 --d_model 128 --d_ff 256 --top_k 3 --train_epochs 100 --patience 10 ...

Table 34: Example of input-output for **tsl** GitHub on **tsl Implementation of classification** task on **Time-series** domain. The README URL is <https://github.com/thuml/Time-Series-Library/blob/main/README.md>. The GitHub URL is <https://github.com/thuml/Time-Series-Library>.

README:
vid2vid
...
Pytorch implementation for high-resolution (e.g., 2048x1024) photorealistic video-to-video translation. It can be used for turning semantic label maps into photo-realistic videos, synthesizing people talking from edge maps, or generating human motions from poses. The core of video-to-video translation is image-to-image translation. Some of our work in that space can be found in pix2pixHD and SPADE.
...
Getting Started
...
Oracle Segment:
...
To test the model (bash ./scripts/face/test_512.sh): #!./scripts/face/test_512.sh
python test.py
--name edge2face_512
--dataroot datasets/face/
--dataset_mode face
--input_nc 15
--loadSize 512
--use_single_G
The test results will be saved in: ./results/edge2face_512/test_latest/.
Instruction:
We have the task of training the edge2face model on a face dataset. Let's set the resolution size this time to 256. Can you provide assistance in drafting the applicable coding structure?
Arguments Requirements:
model: edge2face
dataset: face
resolution size: 256
argument: g1
Ground Truth Output:
python train.py
--name edge2face_256_g1
--dataroot datasets/face/
--dataset_mode face
--input_nc 15
--loadSize 256
--ngf 64
--max_frames_per_gpu 6
--n_frames_total 12
--niter 20
--niter_decay 20

Table 35: Example of input-output for **vid2vid** GitHub on **vid2vid Implementation of Edge to Face Training** task on **Videos** domain. The README URL is <https://github.com/NVIDIA/vid2vid/blob/master/README.md>. The GitHub URL is <https://github.com/NVIDIA/vid2vid>.

C.4 BM25

Retrieval: In this setting, we apply BM25 retriever to get the instruction-related segment in README files. We set the BM25 retriever’s retrieval span at 10 sentences using the NLTK sentence tokenizer because the average length of an Oracle is 9.5.

C.5 Results Across Different GitHub Repositories

Table 36: This table shows the Pass@5 scores of GPT-4, and Claude on the 1/4 set (*) and GPT 3.5 and Claude on the full set across different GitHub repositories. **Oracle**, **Code** and **BM25** represents Oracle, Code, and Retrieval settings.

Repository	GPT-4*			Claude *			GPT 3.5			Claude		
	Oracle	Code	BM25	Oracle	Code	BM25	Oracle	Code	BM25	Oracle	Code	BM25
DGL	80.00	60.00	60.00	40.00	20.00	80.00	47.62	23.81	23.81	28.57	19.05	14.29
BERT	50.00	50.00	16.67	0.00	80.00	16.67	22.73	13.63	13.63	0.00	4.54	0.00
Lavis	42.86	71.43	42.86	57.14	85.71	14.29	55.56	70.37	51.85	51.85	59.26	29.63
If	100.00	100.00	33.33	100.00	0.00	13.33	71.43	61.90	52.38	71.43	76.19	52.38
vid2vid	50.00	75.00	50.00	0.00	25.00	50.00	92.31	76.92	69.23	76.92	38.46	15.38
ESM	60.00	0.00	80.00	0.00	100.00	20.00	47.06	29.41	58.82	5.88	11.76	11.76
OpenCLIP	66.67	66.67	66.67	66.67	66.67	0.00	63.63	36.36	54.55	63.63	63.63	45.46
TSL	25.00	25.00	0.00	25.00	0.00	0.00	14.29	14.29	0.00	7.14	7.14	0.00
EAP	100.00	80.00	0.00	100.00	20.00	80.00	66.66	70.83	33.33	70.83	83.33	20.83
Py-GAN	0.00	12.50	0.00	0.00	12.50	0.00	6.67	0.00	0.00	0.00	0.00	0.00
Py-IM	0.00	0.00	0.00	0.00	0.00	0.00	20.00	0.00	0.00	0.00	0.00	0.00
Learning3d	0.00	0.00	0.00	25.00	0.00	25.00	23.53	47.06	35.29	17.65	0.00	0.00
muzic	80.00	60.00	40.00	60.00	20.00	20.00	66.67	72.22	61.11	38.89	33.33	33.33
Grounded-SAM	60.00	60.00	20.00	0.00	0.00	0.00	0.00	20.00	0.00	5.00	35.00	10.00
Total	48.53	45.59	27.94	34.25	35.61	20.55	36.92	35.39	22.69	30.38	32.31	16.92

D Contribution

The team was led by Xiangru Tang, Yuliang Liu, and Zefan Cai, who not only ran the experiments and designed the models but also played a role in the manuscript preparation. Yanjun also contributed a lot to paper writing. Yanjun Shao, Junjie Lu, and Yichi Zhang supported executing the experimental work. The construction of the dataset, which included tasks such as data annotation and cleansing, was undertaken by Zexuan Deng, Helan Hu, Kaikai An, Ruijun Huang, Shuzheng Si, Sheng Chen, Haozhe Zhao, and Liang Chen. Advisors, e.g., Yan Wang, Tianyu Liu, Zhiwei Jiang, Baobao Chang, Yin Fang, Yujia Qin, Wangchunshu Zhou, Yilun Zhao, Arman Cohan, and Mark Gerstein significantly contributed through discussions.